

Six Flags Project

David Watson, Matthew Hill, Caleb Short,
Cale Rosenbalm, & Luke Vaughan

Iteration Two

Iteration 2 - Use Cases

Use Case: Manage Customer Issues

Actor: HR Employee

Precondition(s):

- HR system is available
- HR Employee is in system

Main Success Scenario:

1. HR employee selects "Manage Customer Issues" in system
2. System retrieves list of customer issues (aka. complaints, incidents, PR cases)
3. System displays issues to HR employee
4. HR employee selects issue
5. System displays issue details
6. HR employee reviews issue information
7. Employee determines appropriate action (resolve, document, escalate)
8. Employee enters action
9. Employee submits update
10. System validates input
11. System updates issue record (notes, status, solution)
12. System stores updates issue information
13. System confirms update to employee

Alternate Flows:

1a. Employee cancels operation

1. HR exits system
2. No changes made

2a. No issues found

1. System displays "No issues available"
2. Process ends

7a. Issue requires escalation

1. HR selects "escalate" option
2. System sends issue to appropriate department
3. System updates status to "Escalated"
4. Process ends

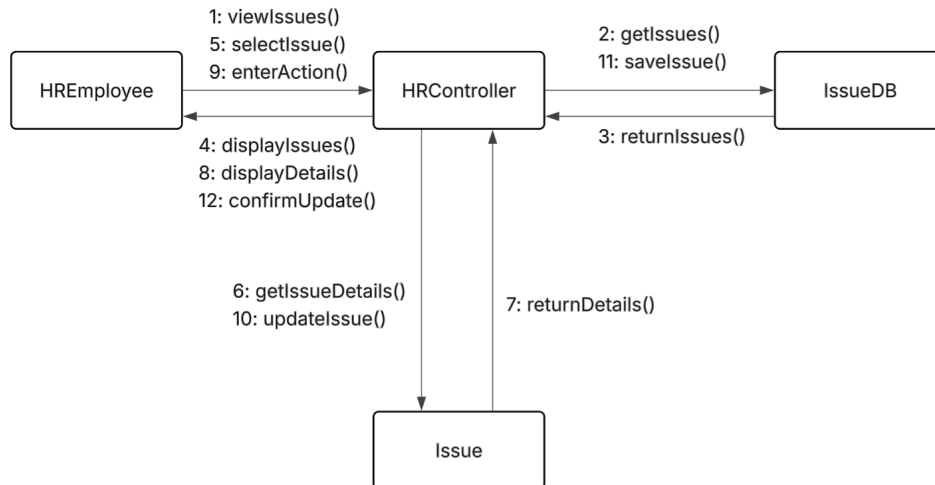
10a. Invalid input

1. System detects missing or incorrect data

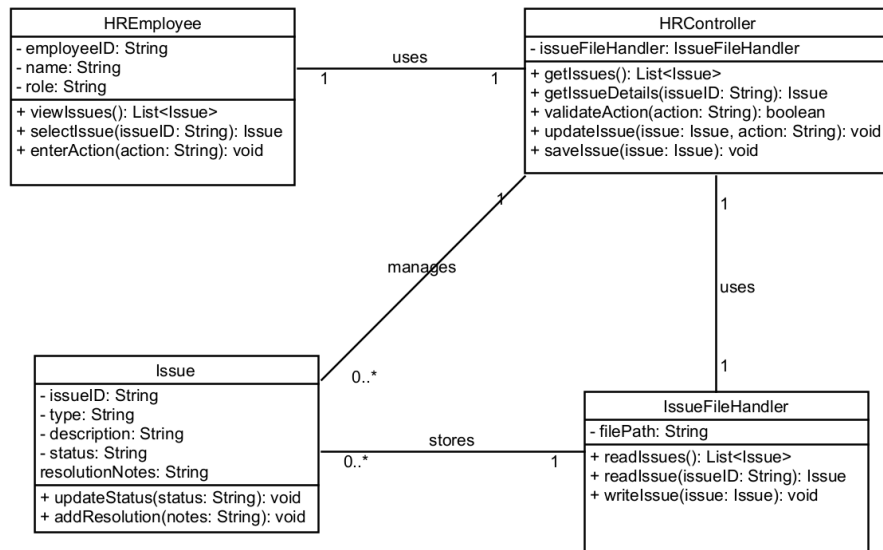
Iteration 2 - Use Cases

2. System displays error message
3. Resume from step 8

Communication Diagram



Partial Class Diagram



Iteration 2 - Use Cases

Use Case: Schedule Ride Maintenance

Actor: Manager

Precondition(s):

- Ride Status is online
- Employee schedule is online
- Manager is logged into system

Main Success Scenario:

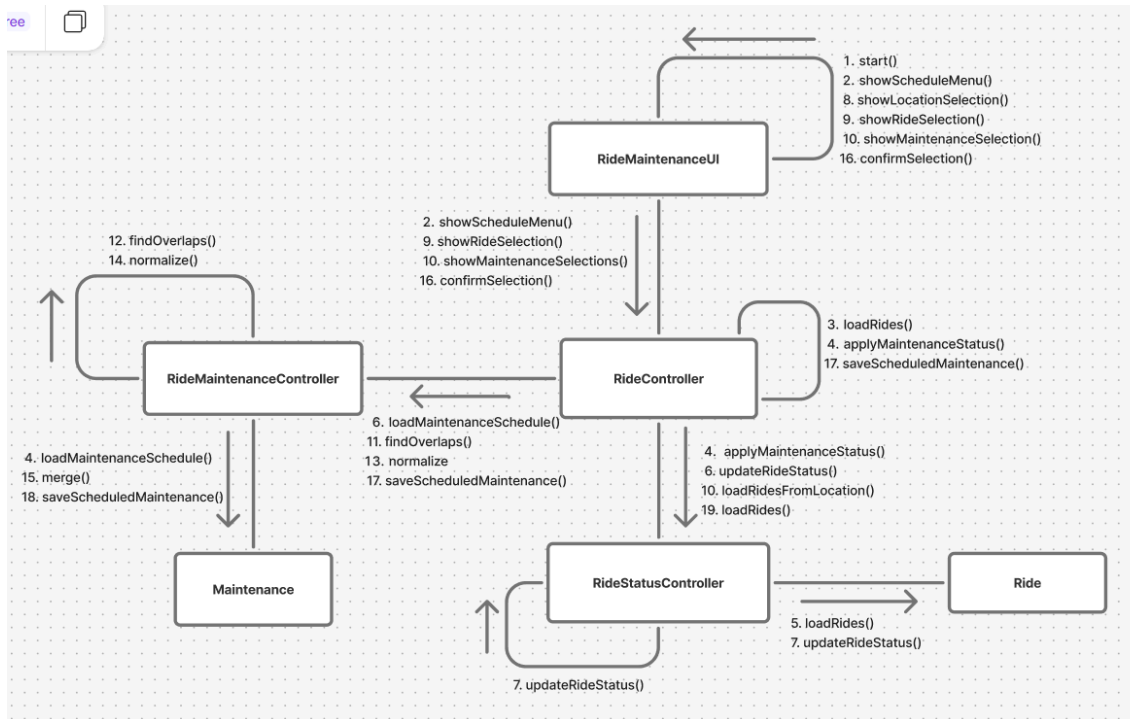
1. Manager selects Park location
2. Manager selects Ride to maintain
3. Manager selects type of maintenance (Daily, One-time, etc.)
4. Manager selects a date/time
5. System records request and outputs information
6. Manager confirms selections
7. System stores maintenance request
8. System updates ride status to "Maintenance" upon date/time selected

Alternate Flows:

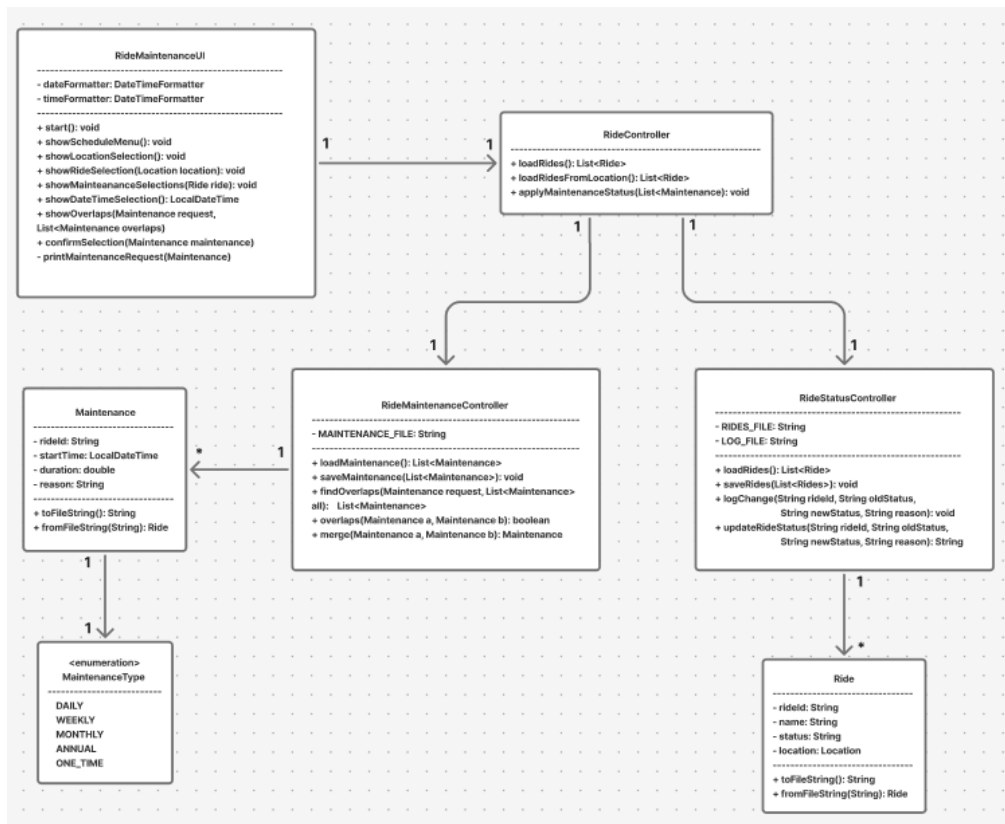
- * Manager cancels request
 - Confirm cancel
 - Delete any data recorded
 - Send user back to Main Menu
- 4a. Overlapping Maintenance Times
 - System informs manager of overlap
 - System prompts manager what they would like to do
 1. Delete Old Request: Remove old request from system
 2. Update Request: Combine time of old and new request
 3. Cancel Request: See *
 - Continue to Step 5

Iteration 2 - Use Cases

Communication Diagram



Partial Class Diagram



Iteration 2 - Use Cases

Use Case: Manage Ride Wait Times

Actor: Ride Operator

Precondition(s):

- Ride exists
- Ride is currently OPEN
- Ride OP is on duty

Main Success Scenario:

1. Ride Operator selects a ride from the system
2. System retrieves current ride information and wait time
3. System displays ride details and current wait time to Ride Operator
4. Ride Operator enters updated wait time
5. System validates the input
6. System updates the ride's wait time
7. System stores updated wait time to file
8. System confirms update to Ride Operator

Alternate Flows:

1a. Ride not found

1. System cannot locate ride in file
2. System displays error
3. Process ends

4a. Invalid wait time entered

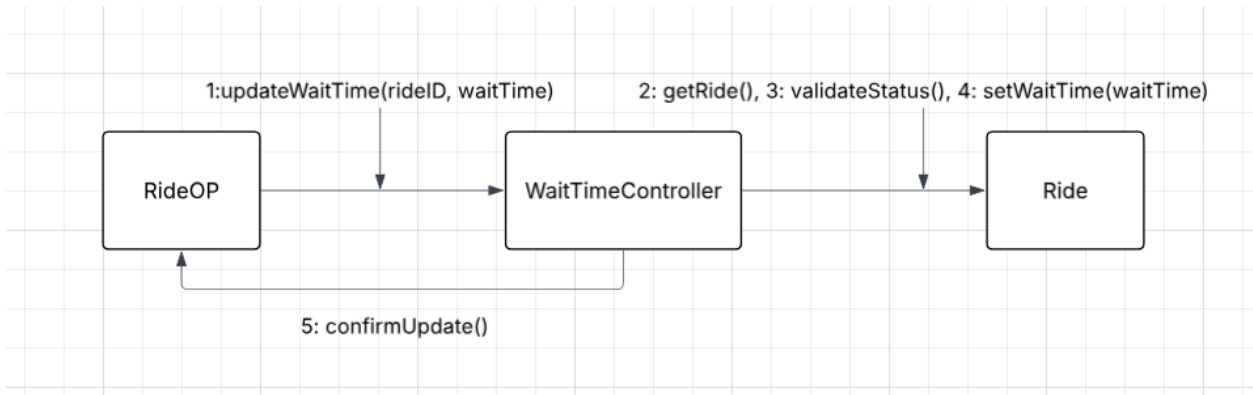
1. System detects input is not a positive number
2. System displays error
3. Resume from step 4

6a. Ride is not OPEN

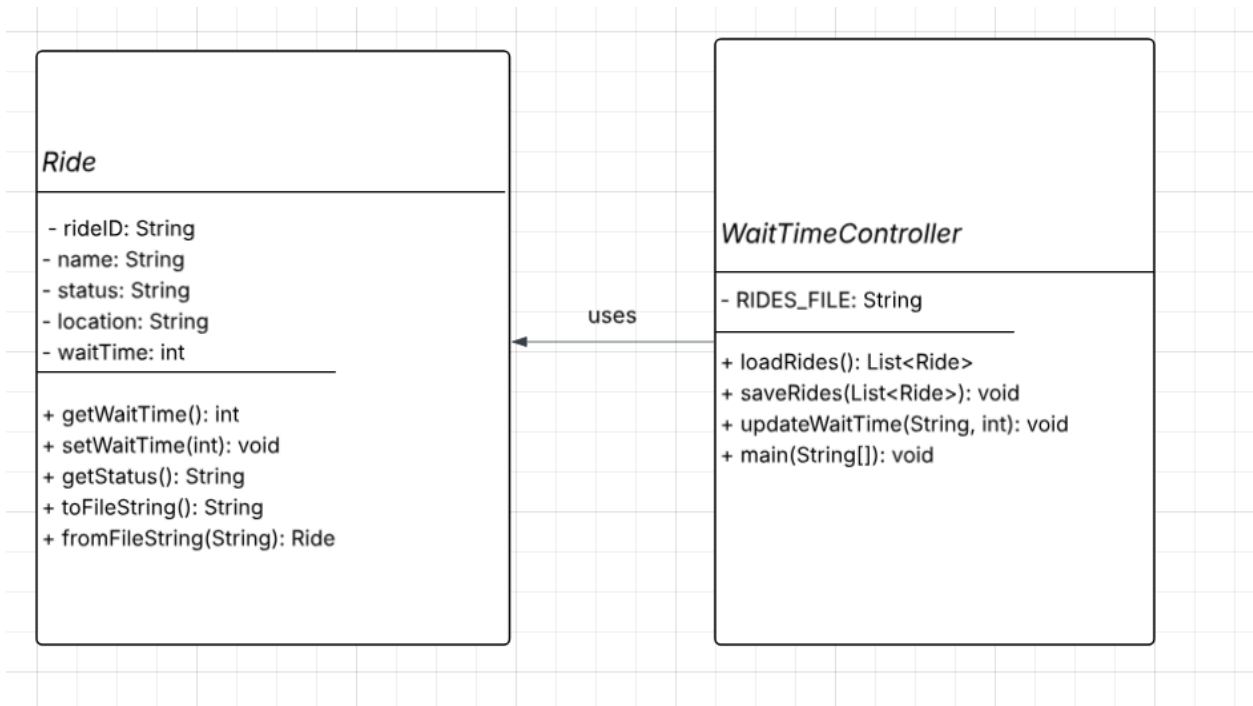
1. System determines ride is closed or under maintenance
2. System displays "Cannot update wait time, ride is not open"
3. Process ends

Iteration 2 - Use Cases

Communication Diagram



Partial Class Diagram



Iteration 2 - Use Cases

Use Case: Hire Employees

Actor: HR

Precondition(s):

- Applicant File Exists
- Employee File Exists
- Applicant record exists in the system
- Approved job opening exists in the system

Main Success Scenario:

1. HR selects option to hire employee
2. System prompts for hiring details
3. HR enters applicant identifier and hiring information
4. HR submits hiring information
5. System validates the entered data and checks applicant and job opening information
6. System creates employee record
7. System writes employee data to employee file
8. System confirms employee has been successfully hired

Alternate Flows:

- 1a. HR cancels operation
 1. HR cancels the process
 2. System stops operation
 3. No employee information is stored
- 5a. Invalid input data
 1. System detects missing or incorrect hiring information
 2. System displays error message
 3. HR is prompted to re-enter information
 4. Resume at step 3
- 5b. Applicant does not exist
 1. System cannot find applicant record
 2. System displays error message
 3. Employee Record is not created
- 5c. Job opening is unavailable
 1. System detects selected job opening is unavailable
 2. System displays error message
 3. HR is prompted to revise hiring information

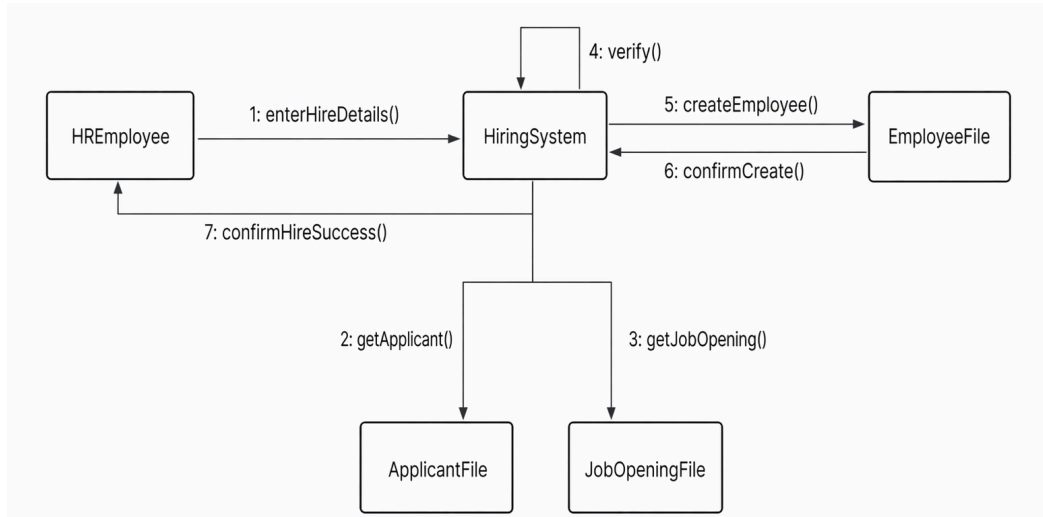
Iteration 2 - Use Cases

4. Resume at step 3

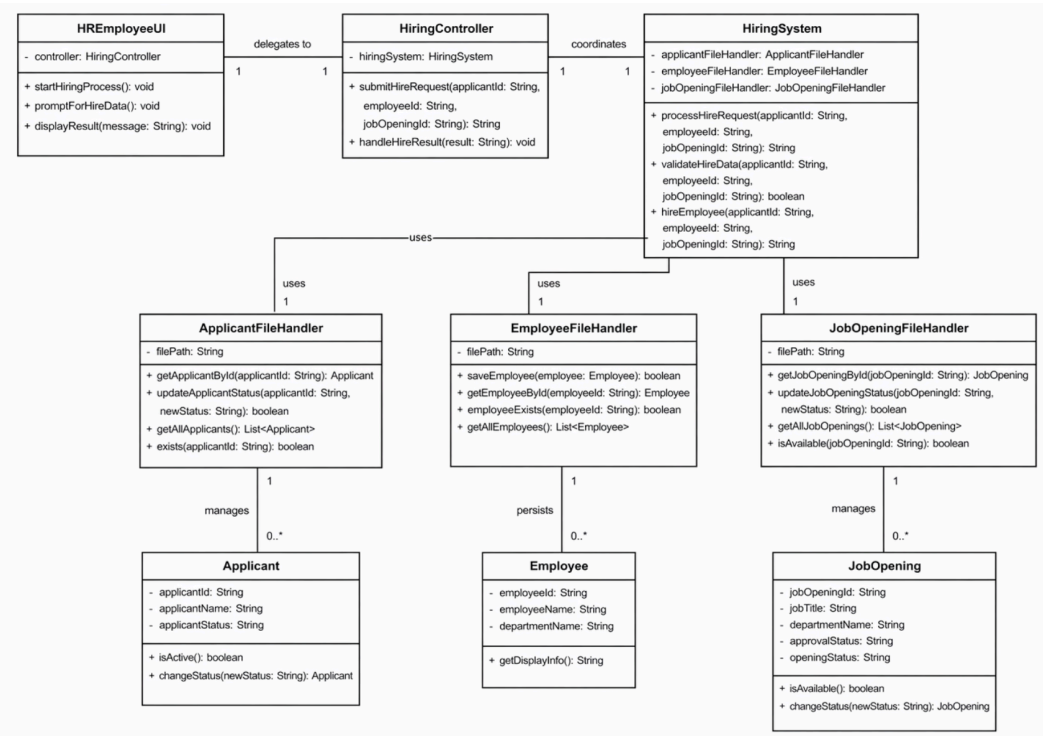
7a. File write error

1. System fails to store employee data in file
2. System displays error message
3. Employee record is not saved

Communication Diagram



Partial Class Diagram



Iteration 2 - Use Cases

Use Case: Process Item Refund

Actor: Guest Services Employee

Precondition(s):

- Refund system is available
- Purchase records files exists
- Inventory file exists

Main Success Scenario:

1. Guest Services Employee selects "Process Refund" in system
2. Employee views list of purchase records and enters the record number to refund
3. System retrieves purchase record
4. System displays purchased items to employee
5. Employee Selects item to refund (Ticket or Inventory Item)
6. Employee enters quantity to refund
7. System validates input
8. System restores item quantity in inventory
9. System logs refund to refund records file
10. System updates purchase record status
11. System confirms refund to employee

Alternate Flows:

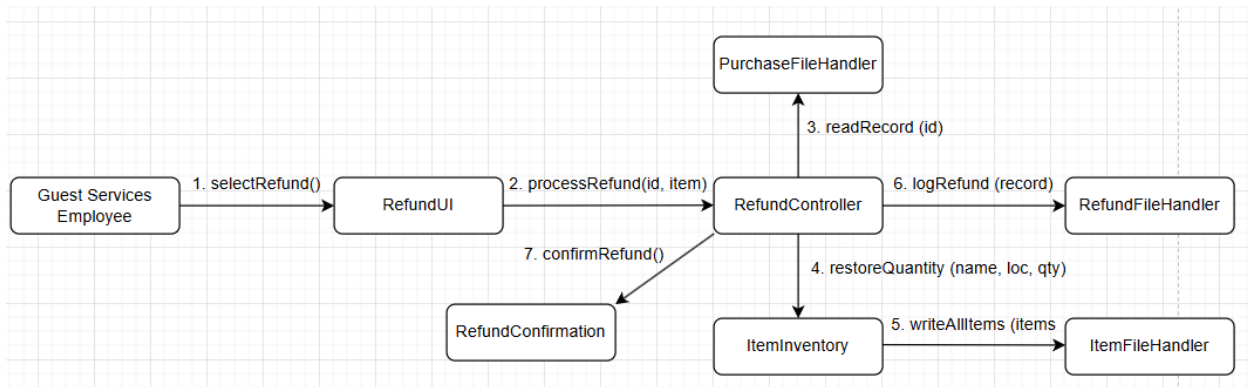
- 2a. Employee cancels operation
 1. System stops operation
 2. No changes made
- 3a. Purchase record not found
 1. System displays "No purchase record found"
 2. Process ends
- 7a. Invalid quantity entered
 1. System detects quantity exceeds purchased amount or is not positive
 2. System displays error message
 3. Resume from step 6
- 8a. Inventory restore fails
 1. System encounters file write error
 2. System displays error message
 3. Process ends

Iteration 2 - Use Cases

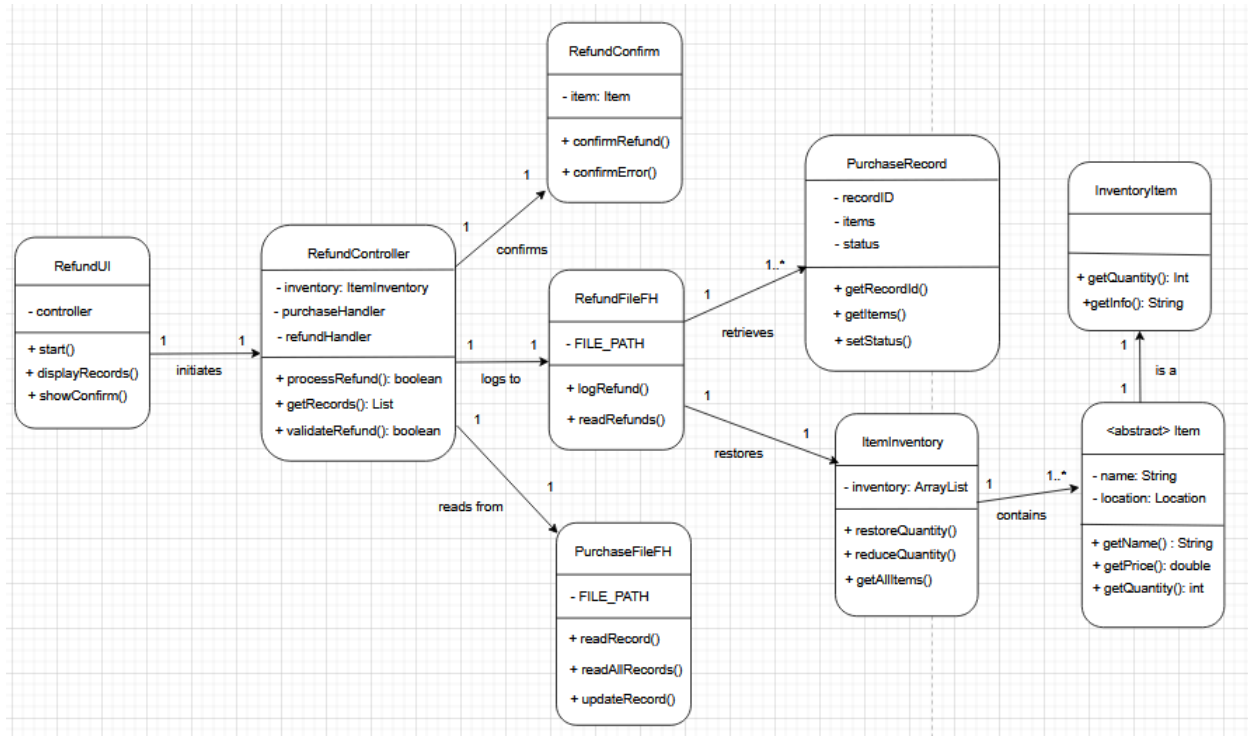
10a. Item is non-refundable

1. System detects item type is marked non-refundable
2. System displays item cannot be refunded”
3. Process ends

Communication Diagram



Partial Class Diagram



Information Expert

1. Manage Customer Issues:

The information expert for Manage Issues is HRController. It knows issues. The HRController is responsible for getIssueDetails(), updateIssue(), and saveIssue(). Therefore, it holds significant knowledge of issues, and is classified as it's information expert.

2. Schedule Ride Maintenance:

ScheduledMaintenance's information expert is the RideController. It knows Maintenance. It holds both controllers that act upon maintenance. It holds a MaintenanceController to save instances to a file using saveMaintenance, and it holds StatusController to update the status with applyMaintenanceStatus() based on the log it knows from the file

3. Manage Ride Wait Times:

The information expert is WaitTimeController, because it's primary responsibility is providing and updating wait times. This requires access to current wait time data for each ride. The WaitTimeController class maintains this information, making it the information expert best suited to manage and update wait times.

Information Expert (contin)

4. Hire Employees:

The information expert of employee hires is the HiringSystem. The HiringSystem knows applicant data best, as seen in its methods processHireRequest, validateHireData, and hireEmployee. In order to process and validate an applicant, HiringSystem must be the information expert with significant information and responsibilities associated with applicants.

5. Process Item Refund:

PurchaseRecord is ItemRefund's information expert. It performs the action of listing purchased items, and this responsibility requires knowledge of the items details of a purchase. The PurchaseRecord stores all purchased items, making it the information expert best suited to provide this data.

Interface List:

The system is organized into logical modules for readability and separation of concerns. These modules do not necessarily reflect physical folder structure

Core Interfaces – Hiring Module

ApplicantFileHandlerInterface

- *Applicant findApplicantById(String applicantId)*
- *boolean updateApplicantStatus(String applicantId, String applicantStatus)*
- *List<Applicant> readAllApplicants()*

EmployeeFileHandlerInterface

- *Employee findEmployeeById(String employeeId)*
- *void writeEmployee(Employee employee)*
- *List<Employee> readAllEmployees()*

HiringControllerInterface

- *String hireEmployee(String applicantId, String employeeId, String jobOpeningId)*

HiringSystemInterface

- *String processHiringRequest(String applicantId, String employeeId, String jobOpeningId)*

Core Interfaces – Item Module

ItemConfirmationInterface

- *boolean validateItem(String itemId)*
- *Item retrieveItemInformation(String id)*
- *boolean isItemValid(Item item)*
- *void verifyItemStatus(Item item)*
- *String getGenMessage()*
- *void setGenMessage(String msg)*
- *Item getItem()*
- *void setItem(Item item)*

Iteration 2 - Interfaces

ItemInterface

- *double getPrice()*
- *String getInfo()*
- *String getName()*
- *int getQuantity()*
- *Location getLocation()*
- *ItemStatus getStatus()*

ItemControllerInterface

- *boolean purchaseItem()*
- *boolean storeItem(String name, String priceInput, String quantityInput, String locationInput)*

ItemInventoryInterface

- *List<String> getItems()*

ItemStatus (Enum)

- *VALID*
- *INVALID*
- *USED*

Core Interfaces – Job Opening, Purchase, Refund, Scheduling & System

JobOpeningFileHandlerInterface

- *JobOpening findJobOpeningById(String jobOpeningId)*
- *boolean updateJobOpeningStatus(String jobOpeningId, String openingStatus)*
- *List<JobOpening> readAllJobOpenings()*

PurchaseFileHandlerInterface

- *PurchaseRecord findRecordById(String recordId)*
- *List<PurchaseRecord> readAllRecords()*
- *boolean updateRecordStatus(String recordId, String status)*
- *boolean updateItemQuantity(String recordId, int itemIndex, int newQuantity)*

Iteration 2 - Interfaces

RefundFileHandlerInterface

- *boolean logRefund(String recordId, String itemName, int quantity)*
- *List<String> readAllRefunds()*

RefundControllerInterface

- *String processRefund(String recordId, int itemIndex, int quantity)*

RefundSystemInterface

- *String processRefundRequest(String recordId, int itemIndex, int quantity)*

ScheduleFileHandlerInterface

- *boolean hasConflict(String employeeId, String scheduleDate, String startTime, String endTime)*
- *void writeSchedule(EmployeeSchedule employeeSchedule)*
- *List<EmployeeSchedule> readAllSchedules()*

SchedulingControllerInterface

- *String manageEmployeeSchedule(String employeeId, String scheduleDate, String startTime, String endTime)*

SchedulingSystemInterface

- *String processSchedulingRequest(String employeeId, String scheduleDate, String startTime, String endTime)*

Enums / System Types

MaintenanceType

- *DAILY*
- *WEEKLY*
- *MONTHLY*
- *ANNUAL*
- *ONE_TIME*

Iteration 2 - Interfaces

Location

- *MAGIC_MOUNTAIN*
- *FIESTA_TEXAS*
- *GREAT_AMERICA*
- *OVER_GEORGIA*
- *GREAT_ESCAPE*

Core System Interface

ParkInterface

- *void run()*

Command Pattern Refactoring Report:

1. AI Software Used for Refactoring

Claude (Anthropic) was used as it provides direct codebase access through its project feature.

2. Prompt(s) Used

- After giving Claude the files required, we first wanted to get more familiar with the Command Pattern and asked:

“Explain what the Command pattern is and what I would need to know/resources to get me familiar with it.”

AI explained what the Command pattern was and how it involved CommandInvoker, and related concrete command classes.

- We started the refactoring by giving Claude details related towards the database with the ability to read our files.

“Here is our Six Flags management system codebase. It currently has two iterations with five team members. Identify every user action across all controllers and UI classes that would qualify as a Command in the Command pattern. Just for reference a Command pattern involves wrapping requests as an object.”

Claude gave the concrete command classes names shown in the outcome part.

- After receiving that list we then asked:
“Now create a Command Interface and CommandInvoker class in a new src/commands package.”... “Generate a concrete Command class for each action you identified, making sure to depend on interfaces rather than concrete implementations where they exist.”

Claude gave the concrete command code shown at the end of the report under “Code”

3. Hand-Holding Required

Moderate hand-holding was needed. The AI initially generated the command classes that depended on concrete controller classes rather than the existing interfaces such as RefundControllerInterface, and HiringControllerInterface. We pointed this out with more information about using all five team members’ use cases and it complied.

4. Outcome

The following was completed: a Command interface with execute(), and getDescription() methods, a CommandInvoker that executes commands and maintains a history log, and 13 concrete command classes covering

Iteration 2 Refactoring Report

all use cases across both iterations including: AddToCartCommand, CheckoutCommand, ProcessRefundCommand, UpdateRideStatusCommand, UpdateWaitTimeCommand, HireEmployeeCommand, ScheduleEmployeeCommand, ScheduleMaintenanceCommand, UpdateIssueCommand, ValidateItemCommand, and StoreItemCommand. What was not completed was wiring the invoker in the UI layer. The UI classes still call controllers directly.

5. Estimate of Remaining Work

We estimate 4 to 6 hours to complete the integration. The UI wiring accounts for most of this as each of the 10 UI classes must replace direct controller calls with command instantiations passed to a shared CommandInvoker. This is mechanical but requires a lot of care to make sure larger UI classes like RideMaintenanceUI is fully implemented. Undo support was not implemented and would require additional state capture per command estimated at 2 or more additional hours if needed. The core pattern structure is complete and correct. The remaining work is substitution.

Iteration 2 Refactoring Report - AI Output

6. Code

Command.java

```
package commands;

/**
 * Command Pattern - Command Interface
 * Every action in the system implements this interface.
 * This allows actions to be executed, undone, and logged
 * uniformly.
 */
public interface Command {
    /**
     * Executes the command.
     * @return null on success, error message on failure
     */
    String execute();

    /**
     * Returns a description of this command for logging purposes.
     */
    String getDescription();
}
```

CommandInvoker.java

```
package commands;

import java.util.ArrayList;
import java.util.List;

/**
 * Command Pattern - Invoker
 * Responsible for executing commands and keeping a history log.
 * The UI classes use this instead of calling controllers
 * directly.
 */
```

Iteration 2 Refactoring Report - AI Output

```
public class CommandInvoker {
    private final List<String> commandHistory = new ArrayList<>();

    /**
     * Executes a command and logs it.
     * @return null on success, error message on failure
     */
    public String invoke(Command command) {
        String result = command.execute();
        if (result == null) {
            commandHistory.add("[SUCCESS] " +
command.getDescription());
        } else {
            commandHistory.add("[FAILED] " +
command.getDescription() + " -> " + result);
        }
        return result;
    }

    public List<String> getHistory() {
        return commandHistory;
    }

    public void printHistory() {
        System.out.println("Command History:");
        for (String entry : commandHistory) {
            System.out.println("  " + entry);
        }
    }
}
```

AddToCartCommand.java

```
package commands;

import controllers.ItemController;
import models.Item;
```

Iteration 2 Refactoring Report - AI Output

```
/**
 * Command Pattern - Concrete Command
 * Wraps the "add item to cart" action.
 */
public class AddToCartCommand implements Command {
    private final ItemController controller;
    private final Item          item;
    private final int           quantity;

    public AddToCartCommand(ItemController controller, Item item,
int quantity) {
        this.controller = controller;
        this.item       = item;
        this.quantity   = quantity;
    }

    @Override
    public String execute() {
        return controller.addToCart(item, quantity);
    }

    @Override
    public String getDescription() {
        return "AddToCart: " + item.getName() + " x" + quantity;
    }
}
```

CheckoutCommand.java

```
package commands;
```

```
import controllers.ItemController;
```

```
/**
 * Command Pattern - Concrete Command
 * Wraps the checkout action.
 */
public class CheckoutCommand implements Command {
    private final ItemController controller;
```

Iteration 2 Refactoring Report - AI Output

```
public CheckoutCommand(ItemController controller) {
    this.controller = controller;
}

@Override
public String execute() {
    return controller.checkout();
}

@Override
public String getDescription() {
    return "Checkout: total $" + String.format("%.2f", controller.calculateTotal());
}
}
```

ProcessFundCommand.java

```
package commands;

import interfaces.RefundControllerInterface;

/**
 * Command Pattern - Concrete Command
 * Wraps the process refund action.
 */
public class ProcessRefundCommand implements Command {
    private final RefundControllerInterface controller;
    private final String recordId;
    private final int itemIndex;
    private final int quantity;

    public ProcessRefundCommand(RefundControllerInterface controller,
                               String recordId, int itemIndex, int quantity) {
        this.controller = controller;
        this.recordId = recordId;
        this.itemIndex = itemIndex;
        this.quantity = quantity;
    }
}
```


Iteration 2 Refactoring Report - AI Output

```
@Override
public String execute() {
    return controller.processRefund(recordId, itemIndex, quantity);
}

@Override
public String getDescription() {
    return "ProcessRefund: record=" + recordId + " itemIndex=" + itemIndex + "
qty=" + quantity;
}
}
```

UpdaterIdStatusCommand.java

```
package commands;

import controllers.RideStatusController;

/**
 * Command Pattern - Concrete Command
 * Wraps the update ride status action.
 */
public class UpdateRideStatusCommand implements Command {
    private final RideStatusController controller;
    private final String rideID;
    private final String newStatus;
    private final String reason;

    public UpdateRideStatusCommand(RideStatusController controller,
                                   String rideID, String newStatus, String reason) {
        this.controller = controller;
        this.rideID = rideID;
        this.newStatus = newStatus;
        this.reason = reason;
    }

    @Override
    public String execute() {
        return controller.updateRideStatus(rideID, newStatus, reason);
    }
}
```

Iteration 2 Refactoring Report - AI Output

```
@Override
public String getDescription() {
    return "UpdateRideStatus: ride=" + rideID + " status=" + newStatus;
}
}
```

UpdateWaitTimeCommand.java

```
package commands;
```

```
import controllers.WaitTimeController;
```

```
/**
 * Command Pattern - Concrete Command
 * Wraps the update wait time action.
 */
```

```
public class UpdateWaitTimeCommand implements Command {
    private final WaitTimeController controller;
    private final String      rideID;
    private final int         waitTime;
```

```
    public UpdateWaitTimeCommand(WaitTimeController controller, String rideID,
int waitTime) {
        this.controller = controller;
        this.rideID     = rideID;
        this.waitTime   = waitTime;
    }
```

```
@Override
public String execute() {
    return controller.updateWaitTime(rideID, waitTime);
}
```

```
@Override
public String getDescription() {
    return "UpdateWaitTime: ride=" + rideID + " waitTime=" + waitTime + "min";
}
}
```

Iteration 2 Refactoring Report - AI Output

HREmployeeCommand.java

```
package commands;

import interfaces.HiringControllerInterface;

/**
 * Command Pattern - Concrete Command
 * Wraps the hire employee action.
 */
public class HireEmployeeCommand implements Command {
    private final HiringControllerInterface controller;
    private final String applicantId;
    private final String employeeId;
    private final String jobOpeningId;

    public HireEmployeeCommand(HiringControllerInterface controller,
                               String applicantId, String employeeId, String jobOpeningId) {
        this.controller = controller;
        this.applicantId = applicantId;
        this.employeeId = employeeId;
        this.jobOpeningId = jobOpeningId;
    }

    @Override
    public String execute() {
        return controller.hireEmployee(applicantId, employeeId, jobOpeningId);
    }

    @Override
    public String getDescription() {
        return "HireEmployee: applicant=" + applicantId + " employee=" +
            employeeId
            + " jobOpening=" + jobOpeningId;
    }
}
```

ScheduleEmployeeCommand.java

```
package commands;
```

Iteration 2 Refactoring Report - AI Output

```
import interfaces.SchedulingControllerInterface;

/**
 * Command Pattern - Concrete Command
 * Wraps the schedule employee action.
 */
public class ScheduleEmployeeCommand implements Command {
    private final SchedulingControllerInterface controller;
    private final String employeeId;
    private final String date;
    private final String startTime;
    private final String endTime;

    public ScheduleEmployeeCommand(SchedulingControllerInterface controller,
                                   String employeeId, String date,
                                   String startTime, String endTime) {
        this.controller = controller;
        this.employeeId = employeeId;
        this.date = date;
        this.startTime = startTime;
        this.endTime = endTime;
    }

    @Override
    public String execute() {
        return controller.manageEmployeeSchedule(employeeId, date, startTime,
        endTime);
    }

    @Override
    public String getDescription() {
        return "ScheduleEmployee: employee=" + employeeId + " date=" + date
        + " " + startTime + "-" + endTime;
    }
}
```

ScheduleMaintenanceCommand.java

```
package commands;
```

Iteration 2 Refactoring Report - AI Output

```
import controllers.RideMaintenanceController;
import models.ScheduledMaintenance;

import java.util.List;

/**
 * Command Pattern - Concrete Command
 * Wraps the schedule maintenance action.
 */
public class ScheduleMaintenanceCommand implements Command {
    private final RideMaintenanceController controller;
    private final List<ScheduledMaintenance> finalState;
    private final String rideID;

    public ScheduleMaintenanceCommand(RideMaintenanceController controller,
                                      List<ScheduledMaintenance> finalState,
                                      String rideID) {
        this.controller = controller;
        this.finalState = finalState;
        this.rideID = rideID;
    }

    @Override
    public String execute() {
        controller.saveScheduledMaintenance(finalState);
        return null;
    }

    @Override
    public String getDescription() {
        return "ScheduleMaintenance: ride=" + rideID;
    }
}
```

UpdateIssueCommand.java

```
package commands;

import controllers.HRController;
```

Iteration 2 Refactoring Report - AI Output

```
import models.Issue;

import java.io.IOException;

/**
 * Command Pattern - Concrete Command
 * Wraps the update customer issue action.
 */
public class UpdateIssueCommand implements Command {
    private final HRController controller;
    private final Issue      issue;
    private final String     action;
    private final String     notes;

    public UpdateIssueCommand(HRController controller, Issue issue,
                              String action, String notes) {
        this.controller = controller;
        this.issue      = issue;
        this.action      = action;
        this.notes       = notes;
    }

    @Override
    public String execute() {
        controller.updateIssue(issue, action, notes);
        try {
            controller.saveIssue(issue);
            return null;
        } catch (IOException e) {
            return "Error saving issue: " + e.getMessage();
        }
    }

    @Override
    public String getDescription() {
        return "UpdateIssue: issue=" + issue.getIssueID() + " action=" + action;
    }
}
```

ValidateItemCommand.java

Iteration 2 Refactoring Report - AI Output

```
package commands;

import models.ItemConfirmation;

/**
 * Command Pattern - Concrete Command
 * Wraps the validate item action.
 */
public class ValidateItemCommand implements Command {
    private final ItemConfirmation itemConfirmation;
    private final String itemId;

    public ValidateItemCommand(ItemConfirmation itemConfirmation, String
itemId) {
        this.itemConfirmation = itemConfirmation;
        this.itemId = itemId;
    }

    @Override
    public String execute() {
        boolean ok = itemConfirmation.validateItem(itemId);
        return ok ? null : itemConfirmation.getGenMessage();
    }

    @Override
    public String getDescription() {
        return "ValidateItem: itemId=" + itemId;
    }
}
```

StoreItemCommand.java

```
package commands;

import controllers.ItemController;
import interfaces.Location;

/**
 * Command Pattern - Concrete Command
```

Iteration 2 Refactoring Report - AI Output

* Wraps the store/add inventory item action.

*/

```
public class StoreItemCommand implements Command {
    private final ItemController controller;
    private final String      name;
    private final double      price;
    private final Location     location;
    private final int          amount;

    public StoreItemCommand(ItemController controller, String name,
                           double price, Location location, int amount) {
        this.controller = controller;
        this.name        = name;
        this.price        = price;
        this.location     = location;
        this.amount       = amount;
    }

    @Override
    public String execute() {
        controller.increaseQuantity(name, price, location, amount);
        return null;
    }

    @Override
    public String getDescription() {
        return "StoreItem: " + name + " x" + amount + " at " + location.name();
    }
}
```


Iteration Two

Software

Iteration 2 Implementation - Models

```
package models;

public class Applicant {
    private final String applicantId;
    private final String applicantName;
    private final String applicantStatus;

    public Applicant(String applicantId, String applicantName) {
        this(applicantId, applicantName, "ACTIVE");
    }

    public Applicant(String applicantId, String applicantName, String
applicantStatus) {
        this.applicantId = applicantId;
        this.applicantName = applicantName;
        this.applicantStatus = applicantStatus;
    }

    public String getApplicantId() { return applicantId; }
    public String getApplicantName() { return applicantName; }
    public String getApplicantStatus() { return applicantStatus; }

    public boolean isActive() {
        return "ACTIVE".equalsIgnoreCase(applicantStatus);
    }

    public Applicant withStatus(String applicantStatus) {
        return new Applicant(applicantId, applicantName,
applicantStatus);
    }

    @Override
    public String toString() {
        return "Applicant{id='" + applicantId + "', name='" +
applicantName
            + "', status='" + applicantStatus + "'}";
    }
}
```

Iteration 2 Implementation - Models

```
package models;

import interfaces.MaintenanceType;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ScheduledMaintenance {
    private static final DateTimeFormatter FORMAT =
DateTimeFormatter.ofPattern("MM/dd/yyyy HH:mm");

    private String rideID;
    private MaintenanceType type;
    private LocalDateTime startTime;
    private double duration;
    private String reason;

    public ScheduledMaintenance(String rideID, MaintenanceType type,
LocalDateTime startTime, double duration, String reason) {
        this.rideID = rideID;
        this.type = type;
        this.startTime = startTime;
        this.duration = duration;
        this.reason = reason;
    }

    public String getRideID() { return rideID; }
    public MaintenanceType getMaintenanceType() { return type; }
    public LocalDateTime getStartTime() { return startTime; }
    public double getDuration() { return duration; }
    public String getReason() { return reason; }

    public String toFileString() {
        return rideID + "," + type + "," + startTime.format(FORMAT) +
", " + duration + "," + reason;
    }

    public static ScheduledMaintenance fromFileString(String line) {
        String[] parts = line.split(",");
        return new ScheduledMaintenance(parts[0],
MaintenanceType.valueOf(parts[1]), LocalDateTime.parse(parts[2],
FORMAT), Double.parseDouble(parts[3]), parts[4]);
    }
}
```

Iteration 2 Implementation - Models

```
package models;

public class Issue {
    private final String issueID;
    private final String type;
    private final String description;
    private String status;
    private String resolutionNotes;

    public Issue(String issueID, String type, String description,
String status, String resolutionNotes) {
        this.issueID          = issueID;
        this.type              = type;
        this.description        = description;
        this.status             = status;
        this.resolutionNotes    = resolutionNotes;
    }

    public String getIssueID()          { return issueID; }
    public String getType()              { return type; }
    public String getDescription()       { return description; }
    public String getStatus()            { return status; }
    public String getResolutionNotes()   { return resolutionNotes; }

    public void updateStatus(String status)      { this.status =
status; }
    public void addResolution(String notes)      {
this.resolutionNotes = notes; }

    public String toFileString() {
        return issueID + "|" + type + "|" + description + "|" + status
+ "|" + resolutionNotes;
    }

    public static Issue fromFileString(String line) {
        String[] p = line.split("\\|", -1);
        return new Issue(p[0].trim(), p[1].trim(), p[2].trim(),
p[3].trim(), p.length > 4 ? p[4].trim() : "");
    }
}
```

Iteration 2 Implementation - Models

```
package models;

public class JobOpening {
    private final String jobOpeningId;
    private final String jobTitle;
    private final String departmentName;
    private final String approvalStatus;
    private final String openingStatus;

    public JobOpening(String jobOpeningId, String jobTitle, String
departmentName,
                        String approvalStatus, String openingStatus) {
        this.jobOpeningId = jobOpeningId;
        this.jobTitle      = jobTitle;
        this.departmentName = departmentName;
        this.approvalStatus = approvalStatus;
        this.openingStatus  = openingStatus;
    }

    public String getJobOpeningId() { return jobOpeningId; }
    public String getJobTitle()     { return jobTitle; }
    public String getDepartmentName() { return departmentName; }
    public String getApprovalStatus() { return approvalStatus; }
    public String getOpeningStatus() { return openingStatus; }

    public boolean isAvailable() {
        return "APPROVED".equalsIgnoreCase(approvalStatus)
            && "OPEN".equalsIgnoreCase(openingStatus);
    }

    public JobOpening withOpeningStatus(String openingStatus) {
        return new JobOpening(jobOpeningId, jobTitle, departmentName,
approvalStatus, openingStatus);
    }

    @Override
    public String toString() {
        return "JobOpening{id='" + jobOpeningId + "', title='" +
jobTitle
            + "', department='" + departmentName + "', approval='" +
approvalStatus
            + "', status='" + openingStatus + "'}";
    }
}
```

Iteration 2 Implementation - Models

```
package models;

import interfaces.Location;

import java.util.ArrayList;
import java.util.List;

public class PurchaseRecord {
    private final String    recordId;
    private final List<Item> items;
    private String          status;

    // Non-refundable item names
    private static final List<String> NON_REFUNDABLE = List.of(
        "Funnel Cake", "Hot Dog", "Burger Combo", "Nachos",
        "Pizza Slice", "Lemonade", "BBQ Sandwich", "Cotton Candy",
        "Popcorn", "Soft Pretzel"
    );

    public PurchaseRecord(String recordId, List<Item> items, String
status) {
        this.recordId = recordId;
        this.items     = new ArrayList<>(items);
        this.status    = status;
    }

    public String    getRecordId() { return recordId; }
    public List<Item> getItems()    { return items; }
    public String    getStatus()    { return status; }
    public void setStatus(String status) { this.status = status; }

    public boolean isRefundable(String itemName) {
        for (String name : NON_REFUNDABLE) {
            if (name.equalsIgnoreCase(itemName)) return false;
        }
        return true;
    }

    // Find item by index (0-based) – avoids duplicate name collision
    public Item findItemByIndex(int index) {
        if (index < 0 || index >= items.size()) return null;
        return items.get(index);
    }

    // Find first item matching name – kept for compatibility
```

Iteration 2 Implementation - Models

```
public Item findItem(String itemName) {
    for (Item item : items) {
        if (item.getName().equalsIgnoreCase(itemName)) return
item;
    }
    return null;
}

public String toFileString() {
    StringBuilder sb = new StringBuilder();
    sb.append(recordId).append("|").append(status).append("|");
    for (int i = 0; i < items.size(); i++) {
        Item item = items.get(i);
        sb.append(item.getName()).append(",")
            .append(item.getPrice()).append(",")
            .append(item.getQuantity()).append(",")
            .append(item.getLocation().name());
        if (i < items.size() - 1) sb.append(";");
    }
    return sb.toString();
}

@Override
public String toString() {
    return "PurchaseRecord{id='" + recordId + "', status='" +
status + "', items=" + items.size() + "}";
}
}
```

Iteration 2 Implementation - Models

```
package models;

public class Ride {
    ...
    private int    waitTime;

    public Ride(String rideID, String name, String status, String
location) {
        ...
        this.waitTime = 0;
    }

    public Ride(String rideID, String name, String status, String
location, int waitTime) {
        ...
        this.waitTime = waitTime;
    }

    ...
    public int    getWaitTime()          { return waitTime; }
    public void    setWaitTime(int waitTime){ this.waitTime =
waitTime; }

    public String toFileString() {
        return rideID + "," + name + "," + status + "," + location +
"," + waitTime;
    }

    public static Ride fromFileString(String line) {
        String[] parts = line.split(",");
        if (parts.length == 5) {
            return new Ride(parts[0], parts[1], parts[2], parts[3],
Integer.parseInt(parts[4].trim()));
        }
        // backwards compatible with old format (no waitTime)
        return new Ride(parts[0], parts[1], parts[2], parts[3]);
    }
}
```


Iteration 2 Implementation - Data

```
package data;

import interfaces.ApplicantFileHandlerInterface;
import models.Applicant;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class ApplicantFileHandler implements
ApplicantFileHandlerInterface {
    private final Path applicantFilePath;

    public ApplicantFileHandler(String applicantFilePath) {
        this.applicantFilePath = Paths.get(applicantFilePath);
    }

    @Override
    public Applicant findApplicantById(String applicantId) throws
IOException {
        for (Applicant applicant : readAllApplicants()) {
            if (applicant.getApplicantId().equals(applicantId)) {
                return applicant;
            }
        }
        return null;
    }

    @Override
    public boolean updateApplicantStatus(String applicantId, String
applicantStatus) throws IOException {
        List<Applicant> applicants = readAllApplicants();
        boolean updated = false;

        for (int i = 0; i < applicants.size(); i++) {
            Applicant applicant = applicants.get(i);
            if (applicant.getApplicantId().equals(applicantId)) {
                applicants.set(i,
applicant.withStatus(applicantStatus));
                updated = true;
            }
        }
    }
}
```

Iteration 2 Implementation - Data

```
        break;
    }
}

if (updated) {
    writeAllApplicants(applicants);
}
return updated;
}

@Override
public List<Applicant> readAllApplicants() throws IOException {
    List<Applicant> applicants = new ArrayList<>();
    if (Files.notExists(applicantFilePath)) return applicants;

    for (String line : Files.readAllLines(applicantFilePath)) {
        if (line.trim().isEmpty()) continue;
        Applicant applicant = parseApplicantRecord(line);
        if (applicant != null) applicants.add(applicant);
    }
    return applicants;
}

private Applicant parseApplicantRecord(String line) {
    String[] parts = line.split("\\|");
    if (parts.length == 2) {
        return new Applicant(parts[0].trim(), parts[1].trim());
    }
    if (parts.length == 3) {
        return new Applicant(parts[0].trim(), parts[1].trim(),
parts[2].trim());
    }
    return null;
}

private void writeAllApplicants(List<Applicant> applicants) throws
IOException {
    if (applicantFilePath.getParent() != null) {
        Files.createDirectories(applicantFilePath.getParent());
    }
    StringBuilder contents = new StringBuilder();
    for (Applicant applicant : applicants) {
        contents.append(formatApplicantRecord(applicant)).append(System.lineS
eparator());
    }
}
```

Iteration 2 Implementation - Data

```
    }
    Files.writeString(applicantFilePath, contents.toString(),
        StandardOpenOption.CREATE,
        StandardOpenOption.TRUNCATE_EXISTING);
}

private String formatApplicantRecord(Applicant applicant) {
    return applicant.getApplicantId() + " | " +
applicant.getApplicantName() + " | "
        + applicant.getApplicantStatus().toUpperCase();
}
}
```

Iteration 2 Implementation - Data

```
package data;

...

public class EmployeeFileHandler implements
EmployeeFileHandlerInterface {

    ...

    @Override
    public void writeEmployee(Employee employee) throws IOException {
        if (employeeFilePath.getParent() != null) {
            Files.createDirectories(employeeFilePath.getParent());
        }
        if (Files.notExists(employeeFilePath)) {
            Files.createFile(employeeFilePath);
        }
        String record = getRecordPrefix() +
formatEmployeeRecord(employee) + System.lineSeparator();
        Files.writeString(employeeFilePath, record,
StandardOpenOption.APPEND);
    }

    ...

    private String formatEmployeeRecord(Employee employee) {
        return employee.getEmployeeId() + " | " +
employee.getEmployeeName() + " | "
        + employee.getDepartmentName();
    }

    private String getRecordPrefix() throws IOException {
        if (Files.size(employeeFilePath) == 0) return "";

        byte[] bytes = Files.readAllBytes(employeeFilePath);
        byte lastByte = bytes[bytes.length - 1];
        if (lastByte == '\n' || lastByte == '\r') return "";
        return System.lineSeparator();
    }
}
```

Iteration 2 Implementation - Data

```
package data;

import models.Issue;

import java.io.*;
import java.nio.file.*;
import java.util.ArrayList;
import java.util.List;

public class IssueFileHandler {
    private final String filePath;

    public IssueFileHandler(String filePath) {
        this.filePath = filePath;
    }

    public List<Issue> readIssues() throws IOException {
        List<Issue> issues = new ArrayList<>();
        Path path = Paths.get(filePath);
        if (!Files.exists(path)) return issues;
        for (String line : Files.readAllLines(path)) {
            if (!line.trim().isEmpty())
                issues.add(Issue.fromFileString(line));
        }
        return issues;
    }

    public Issue readIssue(String issueID) throws IOException {
        for (Issue issue : readIssues()) {
            if (issue.getIssueID().equals(issueID)) return issue;
        }
        return null;
    }

    public void writeIssue(Issue updated) throws IOException {
        List<Issue> issues = readIssues();
        for (int i = 0; i < issues.size(); i++) {
            if
                (issues.get(i).getIssueID().equals(updated.getIssueID())) {
                    issues.set(i, updated);
                    break;
                }
        }
        try (BufferedWriter bw = new BufferedWriter(new
            FileWriter(filePath))) {
```

Iteration 2 Implementation - Data

```
        for (Issue issue : issues) {
            bw.write(issue.toFileString());
            bw.newLine();
        }
    }
}
```

Iteration 2 Implementation - Data

```
package data;

import interfaces.JobOpeningFileHandlerInterface;
import models.JobOpening;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class JobOpeningFileHandler implements
JobOpeningFileHandlerInterface {
    private final Path jobOpeningFilePath;

    public JobOpeningFileHandler(String jobOpeningFilePath) {
        this.jobOpeningFilePath = Paths.get(jobOpeningFilePath);
    }

    @Override
    public JobOpening findJobOpeningById(String jobOpeningId) throws
IOException {
        for (JobOpening jobOpening : readAllJobOpenings()) {
            if (jobOpening.getJobOpeningId().equals(jobOpeningId)) {
                return jobOpening;
            }
        }
        return null;
    }

    @Override
    public boolean updateJobOpeningStatus(String jobOpeningId, String
openingStatus) throws IOException {
        List<JobOpening> jobOpenings = readAllJobOpenings();
        boolean updated = false;

        for (int i = 0; i < jobOpenings.size(); i++) {
            JobOpening jobOpening = jobOpenings.get(i);
            if (jobOpening.getJobOpeningId().equals(jobOpeningId)) {
                jobOpenings.set(i,
jobOpening.withOpeningStatus(openingStatus));
                updated = true;
                break;
            }
        }
    }
}
```

Iteration 2 Implementation - Data

```
        }
    }

    if (updated) {
        writeAllJobOpenings(jobOpenings);
    }
    return updated;
}

@Override
public List<JobOpening> readAllJobOpenings() throws IOException {
    List<JobOpening> jobOpenings = new ArrayList<>();
    if (Files.notExists(jobOpeningFilePath)) return jobOpenings;

    for (String line : Files.readAllLines(jobOpeningFilePath)) {
        if (line.trim().isEmpty()) continue;
        JobOpening jobOpening = parseJobOpeningRecord(line);
        if (jobOpening != null) jobOpenings.add(jobOpening);
    }
    return jobOpenings;
}

private JobOpening parseJobOpeningRecord(String line) {
    String[] parts = line.split("\\|");
    if (parts.length != 5) return null;
    return new JobOpening(
        parts[0].trim(), parts[1].trim(), parts[2].trim(),
        parts[3].trim(), parts[4].trim()
    );
}

private void writeAllJobOpenings(List<JobOpening> jobOpenings)
throws IOException {
    if (jobOpeningFilePath.getParent() != null) {
        Files.createDirectories(jobOpeningFilePath.getParent());
    }
    StringBuilder contents = new StringBuilder();
    for (JobOpening jobOpening : jobOpenings) {
        contents.append(formatJobOpeningRecord(jobOpening)).append(System.lin
eSeparator());
    }
    Files.writeString(jobOpeningFilePath, contents.toString(),
        StandardOpenOption.CREATE,
        StandardOpenOption.TRUNCATE_EXISTING);
}
```


Iteration 2 Implementation - Data

```
}

private String formatJobOpeningRecord(JobOpening jobOpening) {
    return jobOpening.getJobOpeningId() + " | " +
jobOpening.getJobTitle() + " | "
        + jobOpening.getDepartmentName() + " | "
        + jobOpening.getApprovalStatus().toUpperCase() + " | "
        + jobOpening.getOpeningStatus().toUpperCase();
}
}
```

Iteration 2 Implementation - Data

```
package data;

import interfaces.Location;
import interfaces.PurchaseFileHandlerInterface;
import models.InventoryItem;
import models.Item;
import models.PurchaseRecord;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class PurchaseFileHandler implements
PurchaseFileHandlerInterface {
    private final Path purchaseFilePath;

    public PurchaseFileHandler(String purchaseFilePath) {
        this.purchaseFilePath = Paths.get(purchaseFilePath);
    }

    @Override
    public PurchaseRecord findRecordById(String recordId) throws
IOException {
        for (PurchaseRecord record : readAllRecords())
            if (record.getRecordId().equals(recordId)) return record;
        return null;
    }

    @Override
    public List<PurchaseRecord> readAllRecords() throws IOException {
        List<PurchaseRecord> records = new ArrayList<>();
        if (Files.notExists(purchaseFilePath)) return records;

        for (String line : Files.readAllLines(purchaseFilePath)) {
            if (line.trim().isEmpty()) continue;
            PurchaseRecord record = parseRecord(line);
            if (record != null) records.add(record);
        }
        return records;
    }
}
```

Iteration 2 Implementation - Data

```
@Override
public boolean updateRecordStatus(String recordId, String status)
throws IOException {
    List<PurchaseRecord> records = readAllRecords();
    boolean updated = false;
    for (PurchaseRecord record : records) {
        if (record.getRecordId().equals(recordId)) {
            record.setStatus(status);
            updated = true;
            break;
        }
    }
    if (updated) writeAllRecords(records);
    return updated;
}
```

```
@Override
public boolean updateItemQuantity(String recordId, int itemIndex,
int newQuantity) throws IOException {
    List<PurchaseRecord> records = readAllRecords();
    boolean updated = false;
    for (PurchaseRecord record : records) {
        if (record.getRecordId().equals(recordId)) {
            List<Item> items = record.getItems();
            if (itemIndex >= 0 && itemIndex < items.size()) {
                Item item = items.get(itemIndex);
                items.set(itemIndex, new InventoryItem(
                    item.getName(), item.getPrice(),
item.getLocation(), newQuantity));
                updated = true;
            }
            break;
        }
    }
    if (updated) writeAllRecords(records);
    return updated;
}
```

```
public boolean writeRecord(PurchaseRecord record) throws
IOException {
    if (purchaseFilePath.getParent() != null) {
        Files.createDirectories(purchaseFilePath.getParent());
    }
    if (Files.notExists(purchaseFilePath)) {
        Files.createFile(purchaseFilePath);
    }
}
```

Iteration 2 Implementation - Data

```
    }
    String line = record.toFileString() + System.lineSeparator();
    Files.writeString(purchaseFilePath, line,
StandardOpenOption.APPEND);
    return true;
}

private void writeAllRecords(List<PurchaseRecord> records) throws
IOException {
    StringBuilder contents = new StringBuilder();
    for (PurchaseRecord record : records) {

contents.append(record.toFileString()).append(System.lineSeparator())
;
        }
        Files.writeString(purchaseFilePath, contents.toString(),
            StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING);
    }

private PurchaseRecord parseRecord(String line) {
    // Format:
recordId|status|name,price,qty,location;name,price,qty,location
    String[] parts = line.split("\\|");
    if (parts.length != 3) return null;
    String recordId = parts[0].trim();
    String status = parts[1].trim();
    String itemsPart = parts[2].trim();

    List<Item> items = new ArrayList<>();
    if (!itemsPart.isEmpty()) {
        for (String itemStr : itemsPart.split(";")) {
            String[] fields = itemStr.split(",");
            if (fields.length == 4) {
                items.add(new InventoryItem(
                    fields[0].trim(),
                    Double.parseDouble(fields[1].trim()),
                    Location.valueOf(fields[3].trim()),
                    Integer.parseInt(fields[2].trim())
                ));
            }
        }
    }
    return new PurchaseRecord(recordId, items, status);
}
```

Iteration 2 Implementation - Data

```
}  
package data;  
  
import interfaces.RefundFileHandlerInterface;  
import java.io.IOException;  
import java.nio.file.Files;  
import java.nio.file.Path;  
import java.nio.file.Paths;  
import java.nio.file.StandardOpenOption;  
import java.time.LocalDateTime;  
import java.util.ArrayList;  
import java.util.List;  
  
public class RefundFileHandler implements RefundFileHandlerInterface  
{  
    private final Path refundFilePath;  
  
    public RefundFileHandler(String refundFilePath) {  
        this.refundFilePath = Paths.get(refundFilePath);  
    }  
  
    @Override  
    public boolean logRefund(String recordId, String itemName, int  
quantity) throws IOException {  
        if (refundFilePath.getParent() != null)  
            Files.createDirectories(refundFilePath.getParent());  
        if (Files.notExists(refundFilePath))  
            Files.createFile(refundFilePath);  
  
        String entry = recordId + "|" + itemName + "|" + quantity +  
        "|" + LocalDateTime.now() + System.lineSeparator();  
        Files.writeString(refundFilePath, entry,  
StandardOpenOption.APPEND);  
        return true;  
    }  
  
    @Override  
    public List<String> readAllRefunds() throws IOException {  
        List<String> refunds = new ArrayList<>();  
        if (Files.notExists(refundFilePath)) return refunds;  
  
        for (String line : Files.readAllLines(refundFilePath)) {  
            if (!line.trim().isEmpty()) refunds.add(line);  
        }  
        return refunds;  
    }  
}
```

Iteration 2 Implementation - Data

```
}  
}
```

Iteration 2 Implementation - ENUM

```
package interfaces;

import java.time.LocalDate;

public enum MaintenanceType {
    DAILY,
    WEEKLY,
    MONTHLY,
    ANNUAL,
    ONE_TIME;
}
```

Iteration 2 Implementation - Controllers

```
package controllers;

import interfaces.HiringControllerInterface;
import interfaces.HiringSystemInterface;

public class HiringController implements HiringControllerInterface {
    private final HiringSystemInterface hiringSystem;

    public HiringController(HiringSystemInterface hiringSystem) {
        this.hiringSystem = hiringSystem;
    }

    @Override
    public String hireEmployee(String applicantId, String employeeId,
String jobOpeningId) {
        return hiringSystem.processHiringRequest(applicantId,
employeeId, jobOpeningId);
    }
}
```


Iteration 2 Implementation - Controllers

```
package controllers;

import interfaces.ApplicantFileHandlerInterface;
import interfaces.EmployeeFileHandlerInterface;
import interfaces.HiringSystemInterface;
import interfaces.JobOpeningFileHandlerInterface;
import models.Applicant;
import models.Employee;
import models.JobOpening;

import java.io.IOException;

public class HiringSystem implements HiringSystemInterface {
    private static final String SUCCESS_MESSAGE =
"Employee hired successfully.";
    private static final String INVALID_DATA_MESSAGE =
"Invalid hiring data entered.";
    private static final String APPLICANT_NOT_FOUND_MESSAGE =
"Applicant does not exist.";
    private static final String APPLICANT_UNAVAILABLE_MESSAGE =
"Applicant is not active.";
    private static final String JOB_UNAVAILABLE_MESSAGE = "Job
opening is unavailable.";
    private static final String EMPLOYEE_EXISTS_MESSAGE =
"Employee ID already exists.";
    private static final String FILE_ERROR_MESSAGE =
"Unable to hire employee because of a file error.";

    private final ApplicantFileHandlerInterface applicantFileHandler;
    private final EmployeeFileHandlerInterface employeeFileHandler;
    private final JobOpeningFileHandlerInterface
jobOpeningFileHandler;

    public HiringSystem(ApplicantFileHandlerInterface
applicantFileHandler,
                        EmployeeFileHandlerInterface
employeeFileHandler,
                        JobOpeningFileHandlerInterface
jobOpeningFileHandler) {
        this.applicantFileHandler = applicantFileHandler;
        this.employeeFileHandler = employeeFileHandler;
        this.jobOpeningFileHandler = jobOpeningFileHandler;
    }

    @Override
```

Iteration 2 Implementation - Controllers

```
public String processHiringRequest(String applicantId, String
employeeId, String jobOpeningId) {
    if (!validateHiringData(applicantId, employeeId,
jobOpeningId)) {
        return INVALID_DATA_MESSAGE;
    }

    applicantId = applicantId.trim();
    employeeId = employeeId.trim();
    jobOpeningId = jobOpeningId.trim();

    try {
        Applicant applicant =
applicantFileHandler.findApplicantById(applicantId);
        if (applicant == null) return APPLICANT_NOT_FOUND_MESSAGE;
        if (!applicant.isActive()) return
APPLICANT_UNAVAILABLE_MESSAGE;

        if (employeeFileHandler.findEmployeeById(employeeId) !=
null) {
            return EMPLOYEE_EXISTS_MESSAGE;
        }

        JobOpening jobOpening =
jobOpeningFileHandler.findJobOpeningById(jobOpeningId);
        if (jobOpening == null || !jobOpening.isAvailable()) {
            return JOB_UNAVAILABLE_MESSAGE;
        }

        Employee employee = new Employee(
            employeeId, applicant.getApplicantName(),
jobOpening.getDepartmentName());
        employeeFileHandler.writeEmployee(employee);
        if
(!jobOpeningFileHandler.updateJobOpeningStatus(jobOpeningId,
"CLOSED"))
            ||
!applicantFileHandler.updateApplicantStatus(applicantId, "HIRED")) {
            return FILE_ERROR_MESSAGE;
        }
        return SUCCESS_MESSAGE;
    } catch (IOException e) {
        return FILE_ERROR_MESSAGE;
    }
}
```

Iteration 2 Implementation - Controllers

```
private boolean validateHiringData(String applicantId, String
employeeId, String jobOpeningId) {
    return !isBlank(applicantId) && !isBlank(employeeId) &&
!isBlank(jobOpeningId);
}

private boolean isBlank(String value) {
    return value == null || value.trim().isEmpty();
}
}
```

Iteration 2 Implementation - Controllers

```
package controllers;

import data.IssueFileHandler;
import models.Issue;
import java.io.IOException;
import java.util.List;

public class HRController {
    private static final String[] VALID_ACTIONS = {"Resolve",
"Document", "Escalate"};
    private final IssueFileHandler issueFileHandler;

    public HRController(String issuesFilePath) {
        this.issueFileHandler = new IssueFileHandler(issuesFilePath);
    }

    public List<Issue> getIssues() throws IOException {
        return issueFileHandler.readIssues();
    }

    public Issue getIssueDetails(String issueID) throws IOException {
        return issueFileHandler.readIssue(issueID);
    }

    public boolean validateAction(String action) {
        if (action == null || action.isBlank()) return false;
        for (String valid : VALID_ACTIONS)
            if (valid.equalsIgnoreCase(action)) return true;
        return false;
    }

    public void updateIssue(Issue issue, String action, String notes)
    {
        switch (action) {
            case "Resolve"    -> { issue.updateStatus("Resolved");
issue.addResolution(notes); }
            case "Document"  -> { issue.updateStatus("Documented");
issue.addResolution(notes); }
            case "Escalate"  -> { issue.updateStatus("Escalated");
issue.addResolution("Sent to appropriate department"); }
        }
    }

    public void saveIssue(Issue issue) throws IOException {
        issueFileHandler.writeIssue(issue);
    }
}
```

Iteration 2 Implementation - Controllers

```
}  
package controllers;  
...  
  
import data.PurchaseFileHandler;  
import models.PurchaseRecord;  
  
import java.io.IOException;  
import java.util.List;  
import java.util.UUID;  
  
public class PurchaseController {  
    private ItemInventory    inventory;  
    private CartManager      cartManager;  
    private InputValidator   validator;  
    private ItemFactory      factory;  
    private PurchaseFileHandler purchaseFileHandler;  
  
    public PurchaseController(ItemInventory inventory,  
PurchaseFileHandler purchaseFileHandler) {  
        ...  
        this.purchaseFileHandler = purchaseFileHandler;  
    }  
  
    public String checkout() {  
        ...  
  
        String recordId = generateRecordId();  
        List<Item> purchasedItems = new ArrayList<>(cart);  
        PurchaseRecord record = new PurchaseRecord(recordId,  
purchasedItems, "PURCHASED");  
        try {  
            purchaseFileHandler.writeRecord(record);  
        } catch (IOException e) {  
            System.out.println("Warning: Purchase record could not be  
saved: " + e.getMessage());  
        }  
        ...  
    }  
  
    private String generateRecordId() {  
        return "P" + UUID.randomUUID().toString().substring(0,  
7).toUpperCase();  
    }  
}
```

Iteration 2 Implementation - Controllers

```
}  
package controllers;  
  
import interfaces.RefundControllerInterface;  
import interfaces.RefundSystemInterface;  
  
public class RefundController implements RefundControllerInterface {  
    private final RefundSystemInterface refundSystem;  
  
    public RefundController(RefundSystemInterface refundSystem) {  
        this.refundSystem = refundSystem;  
    }  
  
    @Override  
    public String processRefund(String recordId, int itemIndex, int  
quantity) {  
        return refundSystem.processRefundRequest(recordId, itemIndex,  
quantity);  
    }  
}
```

Iteration 2 Implementation - Controllers

```
package controllers;

import data.ItemInventory;
import interfaces.PurchaseFileHandlerInterface;
import interfaces.RefundFileHandlerInterface;
import interfaces.RefundSystemInterface;
import models.Item;
import models.PurchaseRecord;

import java.io.IOException;

public class RefundSystem implements RefundSystemInterface {
    private static final String SUCCESS_MESSAGE = "Refund
processed successfully.";
    private static final String INVALID_DATA_MESSAGE = "Invalid
refund data entered.";
    private static final String RECORD_NOT_FOUND = "No purchase
record found.";
    private static final String ITEM_NOT_FOUND = "Item not
found in purchase record.";
    private static final String NON_REFUNDABLE_MESSAGE = "Item cannot
be refunded.";
    private static final String INVALID_QUANTITY = "Invalid
quantity. Must be greater than zero and not exceed purchased
amount.";
    private static final String FILE_ERROR_MESSAGE = "Unable to
process refund because of a file error.";

    private final PurchaseFileHandlerInterface purchaseFileHandler;
    private final RefundFileHandlerInterface refundFileHandler;
    private final ItemInventory inventory;

    public RefundSystem(PurchaseFileHandlerInterface
purchaseFileHandler,
                        RefundFileHandlerInterface refundFileHandler,
                        ItemInventory inventory) {
        this.purchaseFileHandler = purchaseFileHandler;
        this.refundFileHandler = refundFileHandler;
        this.inventory = inventory;
    }

    @Override
    public String processRefundRequest(String recordId, int itemIndex,
int quantity) {
        if (isBlank(recordId)) return INVALID_DATA_MESSAGE;
```

Iteration 2 Implementation - Controllers

```
try {
    // Step 1: retrieve purchase record
    PurchaseRecord record =
purchaseFileHandler.findRecordById(recordId.trim());
    if (record == null) return RECORD_NOT_FOUND;

    // Step 2: find item by index to avoid duplicate name
collision
    Item item = record.findItemByIndex(itemIndex);
    if (item == null) return ITEM_NOT_FOUND;

    // Step 3: check refundable
    if (!record.isRefundable(item.getName())) return
NON_REFUNDABLE_MESSAGE;

    // Step 4: validate quantity
    if (quantity <= 0 || quantity > item.getQuantity()) return
INVALID_QUANTITY;

    // Step 5: restore inventory for only the refunded item
    inventory.increaseQuantity(item.getName(),
item.getPrice(), item.getLocation(), quantity);

    // Step 6: log refund
    if (!refundFileHandler.logRefund(recordId.trim(),
item.getName(), quantity)) {
        inventory.increaseQuantity(item.getName(),
item.getPrice(), item.getLocation(), -quantity);
        return FILE_ERROR_MESSAGE;
    }

    // Step 7: update item quantity in purchase record
    int newQuantity = item.getQuantity() - quantity;
    purchaseFileHandler.updateItemQuantity(recordId.trim(),
itemIndex, newQuantity);

    // Step 8: update status - REFUNDED only if all items at
qty 0
    boolean allRefunded = record.getItems().stream()
        .allMatch(i -> i == item
            ? newQuantity == 0
            : i.getQuantity() == 0);
    String newStatus = allRefunded ? "REFUNDED" :
"PARTIAL_REFUND";
```


Iteration 2 Implementation - Controllers

```
        purchaseFileHandler.updateRecordStatus(recordId.trim(),
newStatus);

        return SUCCESS_MESSAGE;

    } catch (IOException e) {
        return FILE_ERROR_MESSAGE;
    }
}

private boolean isBlank(String value) {
    return value == null || value.trim().isEmpty();
}
}
```

Iteration 2 Implementation - Controllers

```
package controllers;

import interfaces.MaintenanceType;
import interfaces.Location;
import models.Ride;
import models.ScheduledMaintenance;

import java.io.*;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class RideMaintenanceController {
    private static final String MAINTENANCE_FILE =
"data/rides_maintenance.txt";
    private static final String RIDES_FILE = "data/rides.txt";

    public List<Ride> loadRides() {
        List<Ride> rides = new ArrayList<>();
        File file = new File(RIDES_FILE);
        if (!file.exists()) return rides;
        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.trim().isEmpty())
rides.add(Ride.fromFileString(line));
            }
        } catch (IOException e) {
            System.out.println("Error loading rides: " +
e.getMessage());
        }
        return rides;
    }

    public List<Ride> loadRidesFromLocation(Location location) {
        return loadRides().stream()
            .filter(r ->
r.getLocation().equalsIgnoreCase(location.name().replace("_", " "))
||
r.getLocation().equalsIgnoreCase(location.name()))
            .collect(Collectors.toList());
    }
}
```

Iteration 2 Implementation - Controllers

```
}

public List<ScheduledMaintenance> loadMaintenanceSchedule() {
    List<ScheduledMaintenance> maintenance = new ArrayList<>();
    File file = new File(MAINTENANCE_FILE);
    if (!file.exists()) return maintenance;
    try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
        String line;
        while ((line = br.readLine()) != null) {
            if (!line.trim().isEmpty())
maintenance.add(ScheduledMaintenance.fromFileString(line));
        }
    } catch (IOException e) {
        System.out.println("Error loading maintenance schedule: "
+ e.getMessage());
    }
    return maintenance;
}

public void saveScheduledMaintenance(List<ScheduledMaintenance>
maintenance) {
    try (BufferedWriter bw = new BufferedWriter(new
FileWriter(MAINTENANCE_FILE))) {
        for (ScheduledMaintenance m : maintenance) {
bw.write(m.toFileString()); bw.newLine(); }
    } catch (IOException e) {
        System.out.println("Error saving maintenance: " +
e.getMessage());
    }
}

public List<ScheduledMaintenance>
normalize(List<ScheduledMaintenance> list) {
    List<ScheduledMaintenance> result = new ArrayList<>();
    for (ScheduledMaintenance incoming : list) {
        boolean merged = false;
        for (int i = 0; i < result.size(); i++) {
            ScheduledMaintenance existing = result.get(i);
            if
(!existing.getRideID().equals(incoming.getRideID())) continue;
            if (!overlaps(existing, incoming)) continue;
            if (typeEquals(existing, incoming)) {
                result.set(i, merge(existing, incoming));
                merged = true;
            }
        }
        if (!merged) result.add(incoming);
    }
    return result;
}
```

Iteration 2 Implementation - Controllers

```
        break;
    }
}
if (!merged) result.add(incoming);
}
return result;
}

public List<ScheduledMaintenance>
findOverlaps(ScheduledMaintenance request, List<ScheduledMaintenance>
all) {
    List<ScheduledMaintenance> overlaps = new ArrayList<>();
    for (ScheduledMaintenance m : all) {
        if (!m.getRideID().equals(request.getRideID())) continue;
        if (overlaps(request, m)) overlaps.add(m);
    }
    return overlaps;
}

public boolean overlaps(ScheduledMaintenance a,
ScheduledMaintenance b) {
    LocalDateTime aStart = a.getStartTime().toLocalTime();
    LocalDateTime aEnd    = aStart.plusMinutes((long) (a.getDuration() *
60));
    LocalDateTime bStart = b.getStartTime().toLocalTime();
    LocalDateTime bEnd    = bStart.plusMinutes((long) (b.getDuration() *
60));
    return aStart.isBefore(bEnd) && bStart.isBefore(aEnd);
}

public boolean typeEquals(ScheduledMaintenance a,
ScheduledMaintenance b) {
    return a.getMaintenanceType() == b.getMaintenanceType();
}

public ScheduledMaintenance merge(ScheduledMaintenance a,
ScheduledMaintenance b) {
    LocalDateTime start =
a.getStartTime().isBefore(b.getStartTime()) ? a.getStartTime() :
b.getStartTime();
    LocalDateTime endA =
a.getStartTime().plusMinutes((long) (a.getDuration() * 60));
    LocalDateTime endB =
b.getStartTime().plusMinutes((long) (b.getDuration() * 60));
    LocalDateTime end = endA.isAfter(endB) ? endA : endB;
```

Iteration 2 Implementation - Controllers

```
        double duration = java.time.Duration.between(start,
end).toMinutes() / 60.0;
        return new ScheduledMaintenance(a.getRideID(),
a.getMaintenanceType(), start, duration,
        a.getReason() + " | " + b.getReason());
    }
}
```

Iteration 2 Implementation - Controllers

```
package controllers;

import models.Ride;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class WaitTimeController {
    private static final String RIDES_FILE = "data/rides.txt";

    public List<Ride> loadRides() {
        List<Ride> rides = new ArrayList<>();
        File file = new File(RIDES_FILE);
        if (!file.exists()) return rides;
        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.trim().isEmpty())
rides.add(Ride.fromFileString(line));
            }
        } catch (IOException e) {
            System.out.println("Error loading rides: " +
e.getMessage());
        }
        return rides;
    }

    public void saveRides(List<Ride> rides) {
        try (BufferedWriter bw = new BufferedWriter(new
FileWriter(RIDES_FILE))) {
            for (Ride r : rides) {
                bw.write(r.toFileString());
                bw.newLine();
            }
        } catch (IOException e) {
            System.out.println("Error saving rides: " +
e.getMessage());
        }
    }

    // Returns null on success, error message on failure
    public String updateWaitTime(String rideID, int waitTime) {
        if (waitTime < 0) return "Wait time cannot be negative.";
    }
}
```

Iteration 2 Implementation - Controllers

```
List<Ride> rides = loadRides();
for (Ride r : rides) {
    if (r.getRideID().equals(rideID)) {
        if (!r.getStatus().equals("OPEN"))
            return "Cannot update wait time - ride is not
currently OPEN.";
        r.setWaitTime(waitTime);
        saveRides(rides);
        return null;
    }
}
return "Ride not found in records.";
}
```

Iteration 2 Implementation - User Interface

```
package ui;

import controllers.ItemController;
import data.ItemInventory;

import java.util.Scanner;

public class UserInterface {
    private static final String ANSI_RED    = "\u001B[31m";
    private static final String ANSI_BOLD  = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_YELLOW = "\u001B[33m";

    private final ItemController      controller;
    private final PurchaseUI          purchaseUI;
    private final InventoryUI         inventoryUI;
    private final SchedulingUI        schedulingUI;
    private final ValidationUI         validationUI;
    private final RideStatusUI        rideStatusUI;
    private final RefundUI             refundUI;
    private final HREmployeeUI         hrEmployeeUI;
    private final WaitTimeUI           waitTimeUI;
    private final CustomerIssueUI      customerIssueUI;
    private final RideMaintenanceUI    rideMaintenanceUI;
    private final Scanner              scanner;

    public UserInterface() {
        this.scanner      = new Scanner(System.in);
        this.controller    = new ItemController();
        ItemInventory shared = controller.getSharedInventory();
        this.purchaseUI    = new PurchaseUI(controller, this,
scanner);
        this.inventoryUI   = new InventoryUI(controller, this,
scanner);
        this.schedulingUI  = new SchedulingUI(scanner, this);
        this.validationUI  = new ValidationUI(this, scanner);
        this.rideStatusUI  = new RideStatusUI(this, scanner);
        this.refundUI       = new RefundUI(scanner, this, shared);
        this.hrEmployeeUI  = new HREmployeeUI(this, scanner);
        this.waitTimeUI     = new WaitTimeUI(this, scanner);
        this.customerIssueUI = new CustomerIssueUI(this, scanner);
        this.rideMaintenanceUI = new RideMaintenanceUI(this,
scanner);
    }
}
```


Iteration 2 Implementation - User Interface

```
public void start() {
    showMainMenu();
}

public void showMainMenu() {
    boolean running = true;
    while (running) {
        clearScreen();
        printBanner();
        System.out.println(ANSI_BOLD + "  Main Menu" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  1.  Purchase");
        System.out.println("  2.  Inventory");
        System.out.println("  3.  Employee Scheduling");
        System.out.println("  4.  Hire Employee");
        System.out.println("  5.  Validate Item");
        System.out.println("  6.  Ride Status");
        System.out.println("  7.  Ride Wait Times");
        System.out.println("  8.  Ride Maintenance");
        System.out.println("  9.  Process Refund");
        System.out.println(" 10.  Customer Issues");
        System.out.println(" 11.  Exit");
        System.out.println("  " + "-".repeat(34));
        System.out.print("  Select an option: ");

        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1": purchaseUI.showLocationSelection();
break;
            case "2": inventoryUI.showInventoryMenu();
break;
            case "3": schedulingUI.showSchedulingMenu();
break;
            case "4": hrEmployeeUI.startHiringProcess();
break;
            case "5": validationUI.showValidationMenu();
break;
            case "6": rideStatusUI.showRideMenu();
break;
            case "7": waitTimeUI.showWaitTimeMenu();
break;
            case "8": rideMaintenanceUI.showMaintenanceMenu();
break;
```

Iteration 2 Implementation - User Interface

```
                case "9": refundUI.showRefundMenu();
break;
                case "10": customerIssueUI.showIssueMenu();
break;
                case "11":
                    System.out.println(ANSI_YELLOW + "\n Thanks for
using Six Flags Management System. Goodbye!" + ANSI_RESET);
                    running = false;
                    break;
                default:
                    System.out.println(ANSI_RED + " Invalid option.
Please try again." + ANSI_RESET);
                    pause();
            }
        }
    }

    public void printBanner() {
        System.out.println(ANSI_RED + ANSI_BOLD +
            " +=====+" + ANSI_RESET);
        System.out.println(ANSI_RED + ANSI_BOLD +
            " | |" + ANSI_RESET);
        System.out.println(ANSI_RED + ANSI_BOLD +
            " | S I X F L A G S |" + ANSI_RESET);
        System.out.println(ANSI_YELLOW + ANSI_BOLD +
            " | Management System |" + ANSI_RESET);
        System.out.println(ANSI_RED + ANSI_BOLD +
            " | |" + ANSI_RESET);
        System.out.println(ANSI_RED + ANSI_BOLD +
            " +=====+" + ANSI_RESET);
        System.out.println();
    }

    public void clearScreen() {
        System.out.print("\033[H\033[2J");
        System.out.flush();
    }

    public void pause() {
        System.out.print(" Press Enter to continue...");
        scanner.nextLine();
    }
}
```

Iteration 2 Implementation - User Interface

```
package ui;

import controllers.HRController;
import models.Issue;

import java.io.IOException;
import java.util.List;
import java.util.Scanner;

public class CustomerIssueUI {
    private static final String ISSUES_FILE_PATH = "data/issues.txt";

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";
    private static final String ANSI_YELLOW = "\u001B[33m";

    private final HRController controller;
    private final UserInterface router;
    private final Scanner scanner;

    public CustomerIssueUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
        this.controller = new HRController(ISSUES_FILE_PATH);
    }

    // Step 1: HR selects manage customer issues
    public void showIssueMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Manage Customer Issues"
+ ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  1. View Issues");
            System.out.println("  2. Back");
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": showIssueList(); break;
            }
        }
    }
}
```

Iteration 2 Implementation - User Interface

```
        case "2": running = false; break;
        default:
            System.out.println(ANSI_RED + " Invalid option."
+ ANSI_RESET);
            router.pause();
    }
}

// Step 2 & 3: retrieve and display issues
private void showIssueList() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();

        List<Issue> issues;
        try {
            issues = controller.getIssues();
        } catch (IOException e) {
            System.out.println(ANSI_RED + " Could not load
issues." + ANSI_RESET);
            router.pause();
            return;
        }

        // Alternate flow 2a: no issues found
        if (issues.isEmpty()) {
            System.out.println(" No issues available.");
            router.pause();
            return;
        }

        System.out.println(ANSI_BOLD + " Customer Issues" +
ANSI_RESET);
        System.out.println(" " + "-".repeat(70));
        System.out.printf(" %-5s %-10s %-14s %-35s%n", "No.",
"Issue ID", "Type", "Description");
        System.out.println(" " + "-".repeat(70));
        for (int i = 0; i < issues.size(); i++) {
            Issue issue = issues.get(i);
            String desc = issue.getDescription().length() > 35
                ? issue.getDescription().substring(0, 32) + "..."
                : issue.getDescription();
            System.out.printf(" %-5s %-10s %-14s %-35s%n",
```

Iteration 2 Implementation - User Interface

```
        (i + 1) + ". ",
        issue.getIssueID(),
        issue.getType(),
        desc);
    }
    System.out.println("  " + (issues.size() + 1) + ". Back");
    System.out.println("  " + "-".repeat(70));
    System.out.print("  Select an issue: ");

    String choice = scanner.nextLine().trim();

    // Alternate flow 1a: cancel
    if (isCancel(choice)) return;

    int index;
    try {
        index = Integer.parseInt(choice) - 1;
        if (index == issues.size()) return;
        if (index < 0 || index >= issues.size()) {
            System.out.println(ANSI_RED + "  Invalid
selection." + ANSI_RESET);
            router.pause();
            continue;
        }
    } catch (NumberFormatException e) {
        System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);
        router.pause();
        continue;
    }

    // Step 4 & 5: select issue and display details
    showIssueDetails(issues.get(index));
}

// Step 5 & 6: display issue details
private void showIssueDetails(Issue issue) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "  Issue Details" +
ANSI_RESET);
    System.out.println("  " + "-".repeat(50));
    System.out.println("  ID:          " + issue.getIssueID());
    System.out.println("  Type:          " + issue.getType());
```

Iteration 2 Implementation - User Interface

```
        System.out.println("    Status:      " + issue.getStatus());
        System.out.println("    Description: " +
issue.getDescription());
        System.out.println("    Resolution:  " +
(issue.getResolutionNotes().isEmpty()
    ? "None" : issue.getResolutionNotes()));
        System.out.println("    " + "-".repeat(50));
        System.out.println("    1. Resolve");
        System.out.println("    2. Document");
        System.out.println("    3. Escalate");
        System.out.println("    4. Back");
        System.out.println("    " + "-".repeat(50));
        System.out.print("    Select an action: ");

        String choice = scanner.nextLine().trim();

        // Alternate flow 1a: cancel/back
        if (isCancel(choice) || choice.equals("4")) return;

        String action;
        switch (choice) {
            case "1": action = "Resolve";    break;
            case "2": action = "Document";   break;
            case "3": action = "Escalate";   break;
            default:
                System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
                router.pause();
                return;
        }

        // Alternate flow 7a: escalate goes straight through
        if (action.equals("Escalate")) {
            submitAction(issue, action, "");
            return;
        }

        showNotesInput(issue, action);
    }

    // Step 8 & 9: enter notes and submit
    private void showNotesInput(Issue issue, String action) {
        boolean entering = true;
        while (entering) {
            router.clearScreen();
```

Iteration 2 Implementation - User Interface

```
router.printBanner();
System.out.println(ANSI_BOLD + " " + action + " Issue: "
+ issue.getIssueID() + ANSI_RESET);
System.out.println(" " + "-".repeat(34));
System.out.println(" Enter C to cancel.");
System.out.println(" " + "-".repeat(34));
System.out.print(" Enter notes: ");

String notes = scanner.nextLine().trim();

// Alternate flow 1a
if (isCancel(notes)) return;

// Alternate flow 10a: invalid input
if (notes.isEmpty()) {
    System.out.println(ANSI_RED + " Notes cannot be
empty." + ANSI_RESET);
    router.pause();
    continue;
}

submitAction(issue, action, notes);
entering = false;
}

// Steps 10-13: validate, update, save, confirm
private void submitAction(Issue issue, String action, String
notes) {
    // Step 10: validate
    if (!controller.validateAction(action)) {
        System.out.println(ANSI_RED + " Invalid action." +
ANSI_RESET);
        router.pause();
        return;
    }

    // Step 11: update issue record
    controller.updateIssue(issue, action, notes);

    // Step 12: save to file
    try {
        controller.saveIssue(issue);
    } catch (IOException e) {
```

Iteration 2 Implementation - User Interface

```
        System.out.println(ANSI_RED + "    Error saving issue: " +
e.getMessage() + ANSI_RESET);
        router.pause();
        return;
    }

    // Step 13: confirm
    System.out.println();
    System.out.println(ANSI_GREEN + "    Issue " +
issue.getIssueID() + " updated to: " + issue.getStatus() +
ANSI_RESET);
    router.pause();
}

private boolean isCancel(String value) {
    return value.equalsIgnoreCase("C") ||
value.equalsIgnoreCase("CANCEL");
}
}
```


Iteration 2 Implementation - User Interface

```
package ui;

import controllers.HiringController;
import controllers.HiringSystem;
import data.ApplicantFileHandler;
import data.EmployeeFileHandler;
import data.JobOpeningFileHandler;
import interfaces.HiringControllerInterface;
import models.Applicant;
import models.JobOpening;

import java.util.List;
import java.util.Scanner;

public class HREmployeeUI {
    private static final String APPLICANT_FILE_PATH =
"data/applicants.txt";
    private static final String EMPLOYEE_FILE_PATH =
"data/employees.txt";
    private static final String JOB_OPENING_FILE_PATH =
"data/job_openings.txt";

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private final HiringControllerInterface hiringController;
    private final ApplicantFileHandler applicantFileHandler;
    private final JobOpeningFileHandler jobOpeningFileHandler;
    private final UserInterface router;
    private final Scanner scanner;

    public HREmployeeUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
        this.applicantFileHandler = new
ApplicantFileHandler(APPLICANT_FILE_PATH);
        EmployeeFileHandler efh = new
EmployeeFileHandler(EMPLOYEE_FILE_PATH);
        this.jobOpeningFileHandler = new
JobOpeningFileHandler(JOB_OPENING_FILE_PATH);
        this.hiringController = new HiringController(
            new HiringSystem(applicantFileHandler, efh,
jobOpeningFileHandler));
    }
}
```

Iteration 2 Implementation - User Interface

```
}

// Step 1: start hiring process
public void startHiringProcess() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  HR -- Hire Employee" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  1. Hire Employee");
        System.out.println("  2. Back");
        System.out.println("  " + "-".repeat(34));
        System.out.print("  Select an option: ");

        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1": promptForHireData(); break;
            case "2": running = false;      break;
            default:
                System.out.println(ANSI_RED + "  Invalid option."
+ ANSI_RESET);
                router.pause();
        }
    }
}

// Step 2 & 3: prompt for hire data
private void promptForHireData() {
    boolean entering = true;
    while (entering) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Hire Employee" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(34));

        showAvailableApplicants();
        showAvailableJobOpenings();

        System.out.println();
        System.out.println("  Enter C at any prompt to cancel.");
        System.out.println("  " + "-".repeat(34));
```

Iteration 2 Implementation - User Interface

```
String applicantId = readLine("  Applicant ID: ");
if (isCancel(applicantId)) return;

String employeeId = readLine("  New Employee ID: ");
if (isCancel(employeeId)) return;

String jobOpeningId = readLine("  Job Opening ID: ");
if (isCancel(jobOpeningId)) return;

// Step 4: submit – Step 5: system validates
String result = hiringController.hireEmployee(applicantId,
employeeId, jobOpeningId);
displayResult(result);

    if (result.contains("successfully")
        || result.equals("Applicant does not exist.")
        || result.equals("Unable to hire employee because
of a file error.)) {
        entering = false;
    }
}

// Step 8: display result
public void displayResult(String message) {
    System.out.println();
    if (message.contains("successfully")) {
        System.out.println(ANSI_GREEN + "  " + message +
ANSI_RESET);
    } else {
        System.out.println(ANSI_RED + "  " + message +
ANSI_RESET);
    }
    router.pause();
}

private void showAvailableApplicants() {
    List<Applicant> applicants;
    try {
        applicants = applicantFileHandler.readAllApplicants();
    } catch (Exception e) {
        System.out.println(ANSI_RED + "  Could not load
applicants." + ANSI_RESET);
        return;
    }
}
```

Iteration 2 Implementation - User Interface

```
        System.out.println();
        System.out.println(ANSI_BOLD + "    Available Applicants" +
ANSI_RESET);
        System.out.println("    " + "-".repeat(50));
        System.out.printf("    %-12s %-24s %-10s%n", "ID", "Name",
"Status");
        System.out.println("    " + "-".repeat(50));
        boolean found = false;
        for (Applicant applicant : applicants) {
            if (applicant.isActive()) {
                System.out.printf("    %-12s %-24s %-10s%n",
                    applicant.getApplicantId(),
                    applicant.getApplicantName(),
                    applicant.getApplicantStatus());
                found = true;
            }
        }
        if (!found) System.out.println("    No active applicants
found.");
    }

    private void showAvailableJobOpenings() {
        List<JobOpening> jobOpenings;
        try {
            jobOpenings = jobOpeningFileHandler.readAllJobOpenings();
        } catch (Exception e) {
            System.out.println(ANSI_RED + "    Could not load job
openings." + ANSI_RESET);
            return;
        }

        System.out.println();
        System.out.println(ANSI_BOLD + "    Available Job Openings" +
ANSI_RESET);
        System.out.println("    " + "-".repeat(70));
        System.out.printf("    %-12s %-28s %-22s %-10s%n", "ID",
"Title", "Department", "Status");
        System.out.println("    " + "-".repeat(70));
        boolean found = false;
        for (JobOpening jobOpening : jobOpenings) {
            if (jobOpening.isAvailable()) {
                System.out.printf("    %-12s %-28s %-22s %-10s%n",
                    jobOpening.getJobOpeningId(),
                    jobOpening.getJobTitle(),
```

Iteration 2 Implementation - User Interface

```
        jobOpening.getDepartmentName(),
        jobOpening.getOpeningStatus());
    found = true;
    }
    }
    if (!found) System.out.println("  No available job openings
found.");
    }

    private String readLine(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    private boolean isCancel(String value) {
        return value.equalsIgnoreCase("C") ||
value.equalsIgnoreCase("CANCEL");
    }
}
```

Iteration 2 Implementation - User Interface

```
package ui;

import controllers.RefundController;
import controllers.RefundSystem;
import data.ItemInventory;
import data.PurchaseFileHandler;
import data.RefundFileHandler;
import interfaces.RefundControllerInterface;
import models.Item;
import models.PurchaseRecord;

import java.util.List;
import java.util.Scanner;

public class RefundUI {
    private static final String PURCHASE_FILE_PATH =
"data/purchases.txt";
    private static final String REFUND_FILE_PATH =
"data/refunds.txt";

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private final RefundControllerInterface refundController;
    private final PurchaseFileHandler purchaseFileHandler;
    private final Scanner scanner;
    private final UserInterface router;

    public RefundUI(Scanner scanner, UserInterface router,
ItemInventory sharedInventory) {
        this.scanner = scanner;
        this.router = router;
        PurchaseFileHandler pfh = new
PurchaseFileHandler(PURCHASE_FILE_PATH);
        RefundFileHandler rfh = new
RefundFileHandler(REFUND_FILE_PATH);
        this.purchaseFileHandler = pfh;
        // use the shared inventory instead of creating a new one
        this.refundController = new RefundController(
            new RefundSystem(pfh, rfh, sharedInventory));
    }

    public void showRefundMenu() {
```

Iteration 2 Implementation - User Interface

```
boolean running = true;
while (running) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "  Process Item Refund" +
ANSI_RESET);
    System.out.println("  " + "-".repeat(34));
    System.out.println("  1. Process Refund");
    System.out.println("  2. Back");
    System.out.println("  " + "-".repeat(34));
    System.out.print("  Select an option: ");

    String choice = scanner.nextLine().trim();
    switch (choice) {
        case "1": showRecordList(); break;
        case "2": running = false; break;
        default:
            System.out.println(ANSI_RED + "  Invalid option."
+ ANSI_RESET);
            router.pause();
    }
}

private void showRecordList() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();

        List<PurchaseRecord> records;
        try {
            records = purchaseFileHandler.readAllRecords();
        } catch (Exception e) {
            System.out.println(ANSI_RED + "  Could not load
purchase records." + ANSI_RESET);
            router.pause();
            return;
        }

        if (records.isEmpty()) {
            System.out.println("  No purchase records found.");
            router.pause();
            return;
        }
    }
}
```

Iteration 2 Implementation - User Interface

```
        System.out.println(ANSI_BOLD + "  Purchase Records" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(65));
        System.out.printf("   %-5s %-14s %-16s %-30s%n", "No.",
"Record ID", "Status", "Items");
        System.out.println("  " + "-".repeat(65));
        for (int i = 0; i < records.size(); i++) {
            PurchaseRecord record = records.get(i);
            StringBuilder itemNames = new StringBuilder();
            for (Item item : record.getItems()) {
                if (itemNames.length() > 0) itemNames.append(",
");
                itemNames.append(item.getName()).append("
x").append(item.getQuantity());
            }
            System.out.printf("   %-5s %-14s %-16s %-30s%n",
                (i + 1) + ".",
                record.getRecordId(),
                record.getStatus(),
                itemNames.toString());
        }
        System.out.println("  " + (records.size() + 1) + ".
Back");
        System.out.println("  " + "-".repeat(65));
        System.out.print("  Enter record number: ");

        String choice = scanner.nextLine().trim();
        int index;
        try {
            index = Integer.parseInt(choice) - 1;
            if (index == records.size()) return;
            if (index < 0 || index >= records.size()) {
                System.out.println(ANSI_RED + "  Invalid
selection." + ANSI_RESET);
                router.pause();
                continue;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);
            router.pause();
            continue;
        }
    }
```


Iteration 2 Implementation - User Interface

```
        showItemSelection(records.get(index));
    }
}

private void showItemSelection(PurchaseRecord record) {
    router.clearScreen();
    router.printBanner();
    List<Item> items = record.getItems();

    System.out.println(ANSI_BOLD + "  Record: " +
record.getRecordId() + ANSI_RESET);
    System.out.println("  " + "-".repeat(55));
    System.out.printf("   %-5s %-25s %-10s %-6s%n", "No.", "Name",
"Price", "Qty");
    System.out.println("  " + "-".repeat(55));
    for (int i = 0; i < items.size(); i++) {
        Item item = items.get(i);
        System.out.printf("   %-5s %-25s %-10s %-6s%n",
            (i + 1) + ".",
            item.getName(),
            String.format("$%.2f", item.getPrice()),
            item.getQuantity());
    }
    System.out.println("  " + (items.size() + 1) + ". Cancel");
    System.out.println("  " + "-".repeat(55));
    System.out.print("  Select item to refund: ");

    String choice = scanner.nextLine().trim();
    int itemIndex;
    try {
        itemIndex = Integer.parseInt(choice) - 1;
        if (itemIndex == items.size()) return;
        if (itemIndex < 0 || itemIndex >= items.size()) {
            System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);
            router.pause();
            return;
        }
    } catch (NumberFormatException e) {
        System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);
        router.pause();
        return;
    }
}
```

Iteration 2 Implementation - User Interface

```
        showQuantityInput(record, itemIndex, items.get(itemIndex));
    }

    private void showQuantityInput(PurchaseRecord record, int
itemIndex, Item item) {
        boolean entering = true;
        while (entering) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Refund Item" +
ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  Item:           " + item.getName());
            System.out.println("  Location:           " +
item.getLocation().name().replace("_", " "));
            System.out.println("  Purchased Qty: " +
item.getQuantity());
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Enter quantity to refund (0 to
cancel): ");

            String input = scanner.nextLine().trim();
            int quantity;
            try {
                quantity = Integer.parseInt(input);
                if (quantity == 0) return;
            } catch (NumberFormatException e) {
                System.out.println(ANSI_RED + "  Invalid quantity." +
ANSI_RESET);

                router.pause();
                continue;
            }
            String result = refundController.processRefund(
                record.getRecordId(), itemIndex, quantity);

            if (result.contains("successfully")) {
                System.out.println(ANSI_GREEN + "  " + result +
ANSI_RESET);
            } else {
                System.out.println(ANSI_RED + "  " + result +
ANSI_RESET);
            }
            router.pause();
            entering = false;
        }
    }
```

Iteration 2 Implementation - User Interface

```
}  
}
```

Iteration 2 Implementation - User Interface

```
package ui;

import controllers.RideMaintenanceController;
import interfaces.Location;
import interfaces.MaintenanceType;
import models.Ride;
import models.ScheduledMaintenance;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

public class RideMaintenanceUI {
    private static final String ANSI_BOLD    = "\u001B[1m";
    private static final String ANSI_RESET   = "\u001B[0m";
    private static final String ANSI_RED     = "\u001B[31m";
    private static final String ANSI_GREEN   = "\u001B[32m";
    private static final String ANSI_YELLOW  = "\u001B[33m";

    private final RideMaintenanceController controller;
    private final UserInterface              router;
    private final Scanner                    scanner;
    private final DateTimeFormatter          dateFormatter =
DateTimeFormatter.ofPattern("MM/dd/yyyy");
    private final DateTimeFormatter          timeFormatter =
DateTimeFormatter.ofPattern("H:mm");

    public RideMaintenanceUI(UserInterface router, Scanner scanner) {
        this.router      = router;
        this.scanner      = scanner;
        this.controller = new RideMaintenanceController();
    }

    public void showMaintenanceMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Ride Maintenance" +
ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
        }
    }
}
```

Iteration 2 Implementation - User Interface

```
System.out.println("  1. Schedule Maintenance");
System.out.println("  2. View Maintenance Schedule");
System.out.println("  3. Back");
System.out.println("    " + "-".repeat(34));
System.out.print("  Select an option: ");

String choice = scanner.nextLine().trim();
switch (choice) {
    case "1": showLocationSelection(); break;
    case "2": showScheduleMenu();      break;
    case "3": running = false;         break;
    default:
        System.out.println(ANSI_RED + "    Invalid option."
+ ANSI_RESET);
        router.pause();
    }
}

// Step 1: select location
private void showLocationSelection() {
    Location[] locations = Location.values();
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Select Location" +
ANSI_RESET);
        System.out.println("    " + "-".repeat(34));
        for (int i = 0; i < locations.length; i++) {
            System.out.println("    " + (i + 1) + ". " +
locations[i].name().replace("_", " "));
        }
        System.out.println("    " + (locations.length + 1) + ".
Back");
        System.out.println("    " + "-".repeat(34));
        System.out.print("  Select a location: ");

        String choice = scanner.nextLine().trim();
        int selection;
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
        }
    }
}
```

Iteration 2 Implementation - User Interface

```
        router.pause();
        continue;
    }

    if (selection == locations.length + 1) {
        running = false;
    } else if (selection >= 1 && selection <=
locations.length) {
        showRideSelection(locations[selection - 1]);
    } else {
        System.out.println(ANSI_RED + "   Invalid selection." +
ANSI_RESET);
        router.pause();
    }
}

}

// Step 2: select ride
private void showRideSelection(Location location) {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        List<Ride> rides =
controller.loadRidesFromLocation(location);

        System.out.println(ANSI_BOLD + "   Rides at " +
location.name().replace("_", " ") + ANSI_RESET);
        System.out.println("   " + "-".repeat(40));

        if (rides.isEmpty()) {
            System.out.println("   No rides found at this
location.");
            router.pause();
            return;
        }

        for (int i = 0; i < rides.size(); i++) {
            System.out.println("   " + (i + 1) + ". " +
rides.get(i).getName()
                + " [" + rides.get(i).getStatus() + "]");
        }
        System.out.println("   " + (rides.size() + 1) + ". Back");
        System.out.println("   " + "-".repeat(40));
        System.out.print("   Select a ride: ");
    }
}
```

Iteration 2 Implementation - User Interface

```
String choice = scanner.nextLine().trim();
int selection;
try {
    selection = Integer.parseInt(choice);
} catch (NumberFormatException e) {
    System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
    router.pause();
    continue;
}

if (selection == rides.size() + 1) {
    running = false;
} else if (selection >= 1 && selection <= rides.size()) {
    showMaintenanceTypeSelection(rides.get(selection -
1));
} else {
    System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
    router.pause();
}
}

// Step 3: select maintenance type
private void showMaintenanceTypeSelection(Ride ride) {
    MaintenanceType[] types = MaintenanceType.values();
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "    Maintenance Type for: "
+ ride.getName() + ANSI_RESET);
        System.out.println("    " + "-".repeat(34));
        for (int i = 0; i < types.length; i++) {
            System.out.println("    " + (i + 1) + ". " +
types[i].name());
        }
        System.out.println("    " + (types.length + 1) + ". Back");
        System.out.println("    " + "-".repeat(34));
        System.out.print("    Select type: ");

        String choice = scanner.nextLine().trim();
        int selection;
```

Iteration 2 Implementation - User Interface

```
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
            router.pause();
            continue;
        }

        if (selection == types.length + 1) {
            running = false;
        } else if (selection >= 1 && selection <= types.length) {
            showDateTimeAndDetails(ride, types[selection - 1]);
        } else {
            System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
            router.pause();
        }
    }
}

// Step 4: select date/time and enter details
private void showDateTimeAndDetails(Ride ride, MaintenanceType
type) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "    Schedule Details" +
ANSI_RESET);
    System.out.println("    " + "-".repeat(34));
    System.out.println("    Enter C at any prompt to cancel.");
    System.out.println("    " + "-".repeat(34));

    // Date input
    LocalDate date = null;
    while (date == null) {
        System.out.print("    Start Date (MM/dd/yyyy): ");
        String input = scanner.nextLine().trim();
        if (isCancel(input)) return;
        try {
            date = LocalDate.parse(input, dateFormatter);
            if (date.isBefore(LocalDate.now())) {
                date = null;
                System.out.println(ANSI_RED + "    Date has already
passed. Please try again." + ANSI_RESET);
            }
        }
```


Iteration 2 Implementation - User Interface

```
        } catch (DateTimeParseException e) {
            System.out.println(ANSI_RED + "    Invalid date format.
Please try again." + ANSI_RESET);
        }
    }

    // Time input
    LocalTime time = null;
    while (time == null) {
        System.out.print("    Start Time (H:mm): ");
        String input = scanner.nextLine().trim();
        if (isCancel(input)) return;
        try {
            time = LocalTime.parse(input, timeFormatter);
            if (date.isEqual(LocalDate.now()) &&
time.isBefore(LocalTime.now())) {
                time = null;
                System.out.println(ANSI_RED + "    Time has already
passed. Please try again." + ANSI_RESET);
            }
        } catch (DateTimeParseException e) {
            System.out.println(ANSI_RED + "    Invalid time format.
Please try again." + ANSI_RESET);
        }
    }

    // Duration input
    double duration = 0;
    boolean validDuration = false;
    while (!validDuration) {
        System.out.print("    Duration (hours): ");
        String input = scanner.nextLine().trim();
        if (isCancel(input)) return;
        try {
            duration = Double.parseDouble(input);
            if (duration <= 0) {
                System.out.println(ANSI_RED + "    Duration must be
greater than zero." + ANSI_RESET);
            } else {
                validDuration = true;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
        }
    }
```

Iteration 2 Implementation - User Interface

```
}

// Reason input
System.out.print(" Reason: ");
String reason = scanner.nextLine().trim();
if (isCancel(reason)) return;
if (reason.isEmpty()) {
    System.out.println(ANSI_RED + " Reason cannot be empty."
+ ANSI_RESET);
    router.pause();
    return;
}

LocalDateTime dateTime = LocalDateTime.of(date, time);
ScheduledMaintenance request = new ScheduledMaintenance(
    ride.getRideID(), type, dateTime, duration, reason);

// Step 5: check overlaps
List<ScheduledMaintenance> all =
controller.loadMaintenanceSchedule();
List<ScheduledMaintenance> overlaps =
controller.findOverlaps(request, all);

if (!overlaps.isEmpty()) {
    boolean shouldContinue = showHandleOverlaps(request,
overlaps);
    if (!shouldContinue) return;
}

all.add(request);
List<ScheduledMaintenance> normalized =
controller.normalize(all);

ScheduledMaintenance finalRequest = normalized.stream()
    .filter(m -> m.getRideID().equals(request.getRideID())
        && m.getMaintenanceType() ==
request.getMaintenanceType()
        && controller.overlaps(request, m))
    .findFirst()
    .orElse(request);

// Step 6: confirm
showConfirmation(ride, finalRequest, normalized);
}
```

Iteration 2 Implementation - User Interface

```
// Overlap handling
private boolean showHandleOverlaps(ScheduledMaintenance request,
List<ScheduledMaintenance> overlaps) {
    router.clearScreen();
    router.printBanner();
    boolean sameType = overlaps.stream()
        .anyMatch(x -> x.getMaintenanceType() ==
request.getMaintenanceType());

    System.out.println(ANSI_YELLOW + ANSI_BOLD + "  Overlap
Detected!" + ANSI_RESET);
    System.out.println("  " + "-".repeat(50));
    for (ScheduledMaintenance m : overlaps) {
        System.out.println("    - " + printOverlapSummary(m));
    }
    System.out.println();
    if (sameType) {
        System.out.println(ANSI_YELLOW + "  Same-type overlap:
Request will be auto-merged." + ANSI_RESET);
    } else {
        System.out.println(ANSI_YELLOW + "  Different-type
overlap: Will co-exist." + ANSI_RESET);
    }
    System.out.println("  " + "-".repeat(50));
    System.out.println("    1. Continue");
    System.out.println("    2. Cancel Request");
    System.out.println("  " + "-".repeat(50));
    System.out.print("    Select an option: ");

    String input = scanner.nextLine().trim();
    return input.equals("1");
}

// Step 6 & 7: confirm and save
private void showConfirmation(Ride ride, ScheduledMaintenance
maintenance, List<ScheduledMaintenance> finalState) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "  Confirm Maintenance Request"
+ ANSI_RESET);
    System.out.println("  " + "-".repeat(50));
    System.out.println(formatMaintenanceRequest(ride,
maintenance));
    System.out.println("  " + "-".repeat(50));
    System.out.println("    1. Confirm");
}
```

Iteration 2 Implementation - User Interface

```
System.out.println(" 2. Cancel");
System.out.println(" " + "-".repeat(50));
System.out.print("  Select an option: ");

String choice = scanner.nextLine().trim();
if (choice.equals("1")) {
    controller.saveScheduledMaintenance(finalState);
    System.out.println(ANSI_GREEN + "  Maintenance request
submitted successfully!" + ANSI_RESET);
    router.pause();
}
// cancel just returns to menu
}

// View schedule
private void showScheduleMenu() {
    router.clearScreen();
    router.printBanner();
    List<Ride> rides = controller.loadRides();
    List<ScheduledMaintenance> schedule =
controller.loadMaintenanceSchedule();

    System.out.println(ANSI_BOLD + "  Maintenance Schedule" +
ANSI_RESET);
    System.out.println(" " + "-".repeat(60));

    if (schedule.isEmpty()) {
        System.out.println("  No maintenance requests found.");
    } else {
        for (Location location : Location.values()) {
            System.out.println(ANSI_BOLD + " " +
location.name().replace("_", " ") + ":" + ANSI_RESET);
            for (Ride ride : rides) {
                if
(!ride.getLocation().equalsIgnoreCase(location.name().replace("_", "
")))
                    &&
!ride.getLocation().equalsIgnoreCase(location.name())) continue;
                boolean hasRequests = false;
                for (ScheduledMaintenance m : schedule) {
                    if (m.getRideID().equals(ride.getRideID())) {
                        if (!hasRequests) {
                            System.out.println(" " +
ride.getName() + ":" );
                                hasRequests = true;

```

Iteration 2 Implementation - User Interface

```
    }

    System.out.println(formatMaintenanceRequest(ride, m));
    }
    }
    System.out.println(" " + "-".repeat(60));
    }
    }
    router.pause();
}

private String formatMaintenanceRequest(Ride ride,
ScheduledMaintenance m) {
    String date      = dateFormatter.format(m.getStartTime());
    String startTime = timeFormatter.format(m.getStartTime());
    long minutes     = (long) (m.getDuration() * 60);
    String endTime   =
timeFormatter.format(m.getStartTime().plusMinutes(minutes));

    String s = " " + ride.getLocation() + ": " + ride.getName()
        + " | " + m.getMaintenanceType() + "\n"
        + "  START: " + date + " " + startTime + " - " + endTime +
"\n"
        + "  REASON: " + m.getReason();
    return s;
}

private String printOverlapSummary(ScheduledMaintenance m) {
    String date = dateFormatter.format(m.getStartTime());
    String start = timeFormatter.format(m.getStartTime());
    String end =
timeFormatter.format(m.getStartTime().plusMinutes((long) (m.getDuration() * 60)));
    return m.getMaintenanceType() + " | " + date + " " + start +
"-" + end + " | " + m.getReason();
}

private boolean isCancel(String value) {
    return value.equalsIgnoreCase("C") ||
value.equalsIgnoreCase("CANCEL");
}
}
```

Iteration 2 Implementation - User Interface

```
package ui;

import controllers.WaitTimeController;
import models.Ride;

import java.util.List;
import java.util.Scanner;

public class WaitTimeUI {
    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private final WaitTimeController controller;
    private final UserInterface router;
    private final Scanner scanner;

    public WaitTimeUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
        this.controller = new WaitTimeController();
    }

    public void showWaitTimeMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Manage Ride Wait Times"
+ ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  1. Update Wait Time");
            System.out.println("  2. Back");
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": showRideSelection(); break;
                case "2": running = false;      break;
                default:
                    System.out.println(ANSI_RED + "  Invalid option."
+ ANSI_RESET);

                    router.pause();
            }
        }
    }
}
```

Iteration 2 Implementation - User Interface

```
    }  
    }  
}  
  
private void showRideSelection() {  
    boolean running = true;  
    while (running) {  
        router.clearScreen();  
        router.printBanner();  
  
        List<Ride> rides = controller.loadRides();  
        if (rides.isEmpty()) {  
            System.out.println(ANSI_RED + "    No rides found." +  
ANSI_RESET);  
            router.pause();  
            return;  
        }  
  
        System.out.println(ANSI_BOLD + "    Select a Ride" +  
ANSI_RESET);  
        System.out.println("    " + "-".repeat(65));  
        System.out.printf("    %-4s %-20s %-20s %-12s %-8s%n",  
            "No.", "Name", "Location", "Status", "Wait");  
        System.out.println("    " + "-".repeat(65));  
        for (int i = 0; i < rides.size(); i++) {  
            Ride r = rides.get(i);  
            System.out.printf("    %-4s %-20s %-20s %-12s %-8s%n",  
                (i + 1) + ".",  
                r.getName(),  
                r.getLocation(),  
                r.getStatus(),  
                r.getWaitTime() + " min");  
        }  
        System.out.println("    " + (rides.size() + 1) + ". Back");  
        System.out.println("    " + "-".repeat(65));  
        System.out.print("    Select a ride: ");  
  
        String choice = scanner.nextLine().trim();  
        int selection;  
        try {  
            selection = Integer.parseInt(choice);  
        } catch (NumberFormatException e) {  
            System.out.println(ANSI_RED + "    Invalid input." +  
ANSI_RESET);  
            router.pause();  
        }  
    }  
}
```

Iteration 2 Implementation - User Interface

```
        continue;
    }

    if (selection == rides.size() + 1) {
        running = false;
    } else if (selection >= 1 && selection <= rides.size()) {
        showWaitTimeInput(rides.get(selection - 1));
    } else {
        System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
        router.pause();
    }
}

}

private void showWaitTimeInput(Ride ride) {
    boolean entering = true;
    while (entering) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "    Update Wait Time: " +
ride.getName() + ANSI_RESET);
        System.out.println("    " + "-".repeat(34));
        System.out.println("    Status:          " + ride.getStatus());
        System.out.println("    Current Wait: " + ride.getWaitTime()
+ " min");
        System.out.println("    " + "-".repeat(34));
        System.out.print("    Enter new wait time in minutes (0 to
cancel): ");

        String input = scanner.nextLine().trim();
        int waitTime;
        try {
            waitTime = Integer.parseInt(input);
            if (waitTime == 0) return;
            if (waitTime < 0) {
                System.out.println(ANSI_RED + "    Wait time cannot
be negative." + ANSI_RESET);
                router.pause();
                continue;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "    Invalid input -
please enter a number." + ANSI_RESET);
            router.pause();
        }
    }
}
```


Iteration 2 Implementation - User Interface

```
        continue;
    }

    String error = controller.updateWaitTime(ride.getRideID(),
waitTime);
    if (error != null) {
        System.out.println(ANSI_RED + "  " + error +
ANSI_RESET);
        router.pause();
        entering = false;
    } else {
        System.out.println(ANSI_GREEN + "  Wait time updated
to " + waitTime + " minutes." + ANSI_RESET);
        router.pause();
        entering = false;
    }
}
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
package ui;

import controllers.ItemController;
import interfaces.Location;
import models.Item;

import java.util.ArrayList;
import java.util.Scanner;

public class InventoryUI {
    private final ItemController controller;
    private final UserInterface router;
    private final Scanner scanner;

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private static final String[] LOCATION_NAMES = {
        "Magic Mountain", "Fiesta Texas", "Great America",
        "Over Georgia", "The Great Escape"
    };

    public InventoryUI(ItemController controller, UserInterface
router, Scanner scanner) {
        this.controller = controller;
        this.router = router;
        this.scanner = scanner;
    }

    public void showInventoryMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Inventory Management" +
ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  1. Add Item");
            System.out.println("  2. View Inventory");
            System.out.println("  3. Back");
            System.out.println("  " + "-".repeat(34));
            System.out.print("  Select an option: ");
        }
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
String choice = scanner.nextLine().trim();
switch (choice) {
    case "1": showLocationSelection(); break;
    case "2": showViewInventory();      break;
    case "3": running = false;          break;
    default:
        System.out.println(ANSI_RED + "  Invalid option."
+ ANSI_RESET);
        router.pause();
    }
}

private void showViewInventory() {
    router.clearScreen();
    router.printBanner();
    ArrayList<Item> allItems = controller.getAllItems();
    System.out.println(ANSI_BOLD + "  Current Inventory" +
ANSI_RESET);
    System.out.println("  " + "-".repeat(70));
    System.out.printf("  %-25s %-10s %-10s %-20s%n", "Name",
"Price", "Qty", "Location");
    System.out.println("  " + "-".repeat(70));
    if (allItems.isEmpty()) {
        System.out.println("  No items in inventory.");
    } else {
        for (Item item : allItems) {
            System.out.printf("  %-25s %-10s %-10s %-20s%n",
                item.getName(),
                String.format("%.2f", item.getPrice()),
                item.getQuantity(),
                item.getLocation().name().replace("_", " "));
        }
    }
    System.out.println("  " + "-".repeat(70));
    router.pause();
}

private void showLocationSelection() {
    Location[] locations = Location.values();
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(ANSI_BOLD + "  Add Item -- Select
Location" + ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        for (int i = 0; i < locations.length; i++) {
            System.out.println("  " + (i + 1) + ". " +
LOCATION_NAMES[i]);
        }
        System.out.println("  " + (locations.length + 1) + ".
Back");
        System.out.println("  " + "-".repeat(34));
        System.out.print("  Select a location: ");

        String choice = scanner.nextLine().trim();
        int selection;
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);

            router.pause();
            continue;
        }

        if (selection == locations.length + 1) {
            running = false;
        } else if (selection >= 1 && selection <=
locations.length) {
            showItemTemplates(locations[selection - 1]);
        } else {
            System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);

            router.pause();
        }
    }
}

private void showItemTemplates(Location location) {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        Object[][] templates = controller.getItemTemplates();
        System.out.println(ANSI_BOLD + "  Add Item at " +
location.name().replace("_", " ") + ANSI_RESET);
        System.out.println("  " + "-".repeat(55));
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.printf("  %-4s %-25s %-10s %-10s%n", "No.",
"Name", "Price", "In Stock");
        System.out.println("  " + "-".repeat(55));
        for (int i = 0; i < templates.length; i++) {
            String name = (String) templates[i][0];
            double price = (double) templates[i][1];
            int qty      = controller.getQuantityFor(name,
location);

            System.out.printf("  %-4s %-25s %-10s %-10s%n",
                (i + 1) + ".",
                name,
                String.format("%.2f", price),
                qty);
        }
        System.out.println("  " + (templates.length + 1) + ".
Back");

        System.out.println("  " + "-".repeat(55));
        System.out.print("  Select an item: ");

        String choice = scanner.nextLine().trim();
        int selection;
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);

            router.pause();
            continue;
        }

        if (selection == templates.length + 1) {
            running = false;
        } else if (selection >= 1 && selection <=
templates.length) {
            String name = (String) templates[selection - 1][0];
            double price = (double) templates[selection - 1][1];
            int currentQty = controller.getQuantityFor(name,
location);

            showQuantityInput(name, price, location, currentQty);
        } else {
            System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);

            router.pause();
        }
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
}

    private void showQuantityInput(String name, double price, Location
location, int currentQty) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Add Stock" + ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  Item:           " + name);
        System.out.println("  Current Stock: " + currentQty);
        System.out.println("  " + "-".repeat(34));
        System.out.print("  Enter quantity to add (0 to cancel): ");
        String input = scanner.nextLine().trim();
        int amount;
        try {
            amount = Integer.parseInt(input);
            if (amount == 0) return;
            if (amount < 0) {
                System.out.println(ANSI_RED + "  Quantity must be
greater than zero." + ANSI_RESET);
                router.pause();
                return;
            }
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid number." +
ANSI_RESET);
            router.pause();
            return;
        }

        int newTotal = controller.increaseQuantity(name, price,
location, amount);
        System.out.println(ANSI_GREEN + "  " + name + " updated!
Added: " + amount + " | New total: " + newTotal + ANSI_RESET);
        router.pause();
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
package ui;

import controllers.ItemController;
import interfaces.Location;
import models.Item;

import java.util.ArrayList;
import java.util.Scanner;

public class PurchaseUI {
    private final ItemController controller;
    private final UserInterface router;
    private final Scanner scanner;

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    public PurchaseUI(ItemController controller, UserInterface router,
Scanner scanner) {
        this.controller = controller;
        this.router = router;
        this.scanner = scanner;
    }

    public void showLocationSelection() {
        Location[] locations = Location.values();
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + " Purchase -- Select
Location" + ANSI_RESET);
            System.out.println(" " + "-".repeat(34));
            for (int i = 0; i < locations.length; i++) {
                System.out.println(" " + (i + 1) + ". " +
locations[i].name().replace("_", " "));
            }
            System.out.println(" " + (locations.length + 1) + ".
Back");

            System.out.println(" " + "-".repeat(34));
            System.out.print(" Select a location: ");

            String choice = scanner.nextLine().trim();
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
int selection;
try {
    selection = Integer.parseInt(choice);
} catch (NumberFormatException e) {
    System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
    router.pause();
    continue;
}

if (selection == locations.length + 1) {
    running = false;
} else if (selection >= 1 && selection <=
locations.length) {
    showItemSelection(locations[selection - 1]);
} else {
    System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
    router.pause();
}
}

}

private void showItemSelection(Location location) {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        ArrayList<Item> items =
controller.getAvailableItemsByLocation(location);
        System.out.println(ANSI_BOLD + "    Items at " +
location.name().replace("_", " ") + ANSI_RESET);
        System.out.println("    " + "-".repeat(50));
        if (items.isEmpty()) {
            System.out.println("    No items available at this
location.");
            System.out.println("    " + "-".repeat(50));
            router.pause();
            return;
        }
        System.out.printf("    %-4s %-25s %-10s %-6s%n", "No.",
"Name", "Price", "Qty");
        System.out.println("    " + "-".repeat(50));
        for (int i = 0; i < items.size(); i++) {
            Item item = items.get(i);
```


Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.printf("  %-4s %-25s %-10s %-6s%n",
            (i + 1) + ".",
            item.getName(),
            String.format("%.2f", item.getPrice()),
            item.getQuantity());
    }
    System.out.println("  " + (items.size() + 1) + ". Back");
    System.out.println("  " + "-".repeat(50));
    System.out.print("  Select an item: ");

    String choice = scanner.nextLine().trim();
    int selection;
    try {
        selection = Integer.parseInt(choice);
    } catch (NumberFormatException e) {
        System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);
        router.pause();
        continue;
    }

    if (selection == items.size() + 1) {
        running = false;
    } else if (selection >= 1 && selection <= items.size()) {
        showQuantityInput(items.get(selection - 1));
    } else {
        System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);
        router.pause();
    }
}

private void showQuantityInput(Item item) {
    boolean entering = true;
    while (entering) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Add to Cart" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(34));
        System.out.println("  Item:      " + item.getName());
        System.out.println("  Price:      $" +
String.format("%.2f", item.getPrice()));
        System.out.println("  Available: " + item.getQuantity());
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
System.out.println("  " + "-".repeat(34));
System.out.print("  Enter quantity (0 to cancel): ");

String input = scanner.nextLine().trim();
int quantity;
try {
    quantity = Integer.parseInt(input);
    if (quantity == 0) return;
    if (quantity < 0) {
        System.out.println(ANSI_RED + "  Quantity must be
positive." + ANSI_RESET);
        router.pause();
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println(ANSI_RED + "  Invalid number." +
ANSI_RESET);
    router.pause();
    continue;
}

String error = controller.addToCart(item, quantity);
if (error != null) {
    System.out.println(ANSI_RED + "  Error: " + error +
ANSI_RESET);
    router.pause();
} else {
    System.out.println(ANSI_GREEN + "  Added to cart." +
ANSI_RESET);
    entering = false;
    showCartMenu();
}
}

}

public void showCartMenu() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Cart" + ANSI_RESET);
        System.out.println("  " + "-".repeat(50));
        for (String line : controller.getCartInfo().split("\n")) {
            System.out.println("  " + line);
        }
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(" " + "-".repeat(50));
        System.out.printf(" %-30s $%.2f%n", "Total:",
controller.calculateTotal());
        System.out.println(" " + "-".repeat(50));
        System.out.println(" 1. Continue Shopping");
        System.out.println(" 2. Remove Item");
        System.out.println(" 3. Checkout");
        System.out.println(" 4. Back to Main");
        System.out.println(" " + "-".repeat(50));
        System.out.print(" Select an option: ");

        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1": showLocationSelection(); running = false;
break;
            case "2": showRemoveItem();
break;
            case "3": showCheckout();                running = false;
break;
            case "4":                                running = false;
break;
            default:
                System.out.println(ANSI_RED + " Invalid option."
+ ANSI_RESET);
                router.pause();
        }
    }

    private void showRemoveItem() {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + " Remove Item" + ANSI_RESET);
        System.out.println(" " + "-".repeat(50));
        for (String line : controller.getCartInfo().split("\n")) {
            System.out.println(" " + line);
        }
        System.out.println(" " + "-".repeat(50));
        System.out.print(" Enter item number to remove (0 to cancel):
");

        String input = scanner.nextLine().trim();
        try {
            int index = Integer.parseInt(input) - 1;
            if (index == -1) return;
            controller.removeFromCart(index);
        }
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(ANSI_GREEN + "  Item removed." +
ANSI_RESET);
    } catch (NumberFormatException e) {
        System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);
    }
    router.pause();
}

private void showCheckout() {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "  Checkout" + ANSI_RESET);
    System.out.println("  " + "-".repeat(50));
    if (controller.isCartEmpty()) {
        System.out.println(ANSI_RED + "  Cart is empty." +
ANSI_RESET);
        router.pause();
        return;
    }
    for (String line : controller.getCartInfo().split("\n")) {
        System.out.println("  " + line);
    }
    System.out.println("  " + "-".repeat(50));
    double total = controller.calculateTotal();
    System.out.printf("  %-30s $%.2f\n", "Total:", total);
    System.out.println("  " + "-".repeat(50));
    System.out.print("  Enter card info: ");
    scanner.nextLine();

    String error = controller.checkout();
    if (error != null) {
        System.out.println(ANSI_RED + "  Transaction failed: " +
error + ANSI_RESET);
    } else {
        System.out.println(ANSI_GREEN + "  Purchase successful!" +
ANSI_RESET);
        System.out.printf("  Total charged: $%.2f\n", total);
        System.out.println("  Thank you for your order.");
    }
    router.pause();
}
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
package ui;

import controllers.RideStatusController;
import models.Ride;

import java.util.List;
import java.util.Scanner;

public class RideStatusUI {
    private final UserInterface router;
    private final Scanner scanner;
    private final RideStatusController controller;

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    private static final String[] STATUSES = {"OPEN", "CLOSED",
"MAINTENANCE"};

    public RideStatusUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
        this.controller = new RideStatusController();
    }

    public void showRideMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + " Ride Status Management"
+ ANSI_RESET);
            System.out.println(" " + "-".repeat(34));
            System.out.println(" 1. Update Ride Status");
            System.out.println(" 2. Back");
            System.out.println(" " + "-".repeat(34));
            System.out.print(" Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": showRideSelection(); break;
                case "2": running = false; break;
                default:
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(ANSI_RED + "    Invalid option."
+ ANSI_RESET);
        router.pause();
    }
}

private void showRideSelection() {
    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        List<Ride> rides = controller.loadRides();

        if (rides.isEmpty()) {
            System.out.println(ANSI_RED + "    No rides found." +
ANSI_RESET);
            router.pause();
            return;
        }

        System.out.println(ANSI_BOLD + "    Select a Ride" +
ANSI_RESET);
        System.out.println("    " + "-".repeat(60));
        System.out.printf("    %-4s %-20s %-20s %-12s%n", "No.",
"Name", "Location", "Status");
        System.out.println("    " + "-".repeat(60));
        for (int i = 0; i < rides.size(); i++) {
            Ride ride = rides.get(i);
            System.out.printf("    %-4s %-20s %-20s %-12s%n",
                (i + 1) + ".",
                ride.getName(),
                ride.getLocation(),
                ride.getStatus());
        }
        System.out.println("    " + (rides.size() + 1) + ". Back");
        System.out.println("    " + "-".repeat(60));
        System.out.print("    Select a ride: ");

        String choice = scanner.nextLine().trim();
        int selection;
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
        router.pause();
        continue;
    }

    if (selection == rides.size() + 1) {
        running = false;
    } else if (selection >= 1 && selection <= rides.size()) {
        showUpdateForm(rides.get(selection - 1));
    } else {
        System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
        router.pause();
    }
}

}

private void showUpdateForm(Ride ride) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "    Update Ride: " +
ride.getName() + ANSI_RESET);
    System.out.println("    " + "-".repeat(34));
    System.out.println("    Current Status: " + ride.getStatus());
    System.out.println();
    System.out.println("    New Status:");
    for (int i = 0; i < STATUSES.length; i++) {
        System.out.println("    " + (i + 1) + ". " + STATUSES[i]);
    }
    System.out.println("    " + (STATUSES.length + 1) + ". Cancel");
    System.out.println("    " + "-".repeat(34));
    System.out.print("    Select new status: ");

    String choice = scanner.nextLine().trim();
    int selection;
    try {
        selection = Integer.parseInt(choice);
    } catch (NumberFormatException e) {
        System.out.println(ANSI_RED + "    Invalid input." +
ANSI_RESET);
        router.pause();
        return;
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        if (selection == STATUSES.length + 1) return;

        if (selection < 1 || selection > STATUSES.length) {
            System.out.println(ANSI_RED + "   Invalid selection." +
ANSI_RESET);
            router.pause();
            return;
        }

        String newStatus = STATUSES[selection - 1];
        System.out.print("   Enter reason: ");
        String reason = scanner.nextLine().trim();

        String error = controller.updateRideStatus(ride.getRideID(),
newStatus, reason);
        if (error != null) {
            System.out.println(ANSI_RED + "   Error: " + error +
ANSI_RESET);
        } else {
            System.out.println(ANSI_GREEN + "   " + ride.getName() + "
updated to " + newStatus + "." + ANSI_RESET);
        }
        router.pause();
    }
}
```


Iteration 2 Implementation - ITER 1 Updated User Interface

```
package ui;

import controllers.SchedulingController;
import controllers.SchedulingSystem;
import data.EmployeeFileHandler;
import data.ScheduleFileHandler;
import interfaces.ScheduleFileHandlerInterface;
import interfaces.SchedulingControllerInterface;
import models.Employee;
import models.EmployeeSchedule;

import java.util.List;
import java.util.Scanner;

public class SchedulingUI {
    private static final String EMPLOYEE_FILE_PATH =
"data/employees.txt";
    private static final String SCHEDULE_FILE_PATH =
"data/schedules.txt";

    private static final String ANSI_BOLD    = "\u001B[1m";
    private static final String ANSI_RESET  = "\u001B[0m";
    private static final String ANSI_RED    = "\u001B[31m";
    private static final String ANSI_GREEN  = "\u001B[32m";

    private final SchedulingControllerInterface schedulingController;
    private final ScheduleFileHandlerInterface scheduleFileHandler;
    private final EmployeeFileHandler          employeeFileHandler;
    private final UserInterface                router;
    private final Scanner                      scanner;

    public SchedulingUI(Scanner scanner, UserInterface router) {
        this.scanner          = scanner;
        this.router           = router;
        this.employeeFileHandler = new
EmployeeFileHandler(EMPLOYEE_FILE_PATH);
        this.scheduleFileHandler = new
ScheduleFileHandler(SCHEDULE_FILE_PATH);
        this.schedulingController = new SchedulingController(
            new SchedulingSystem(employeeFileHandler,
scheduleFileHandler));
    }

    public void showSchedulingMenu() {
        boolean running = true;
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
while (running) {
    router.clearScreen();
    router.printBanner();
    System.out.println(ANSI_BOLD + "  Employee Scheduling" +
ANSI_RESET);
    System.out.println("  " + "-".repeat(34));
    System.out.println("  1. Schedule Employee");
    System.out.println("  2. View Schedules");
    System.out.println("  3. Back");
    System.out.println("  " + "-".repeat(34));
    System.out.print("  Select an option: ");

    String choice = scanner.nextLine().trim();
    switch (choice) {
        case "1": showEmployeeSelection(); break;
        case "2": showViewSchedules();      break;
        case "3": running = false;          break;
        default:
            System.out.println(ANSI_RED + "  Invalid option.
Please try again." + ANSI_RESET);
            router.pause();
    }
}

private void showEmployeeSelection() {
    List<Employee> employees;
    try {
        employees = employeeFileHandler.readAllEmployees();
    } catch (Exception e) {
        System.out.println(ANSI_RED + "  Could not load
employees." + ANSI_RESET);
        router.pause();
        return;
    }

    if (employees.isEmpty()) {
        System.out.println("  No employees found.");
        router.pause();
        return;
    }

    router.clearScreen();
    router.printBanner();
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        System.out.println(ANSI_BOLD + "  Select Employee" +
ANSI_RESET);
        System.out.println("  " + "-".repeat(50));
        System.out.printf("    %-4s %-25s %-20s%n", "No.", "Name",
"Department");
        System.out.println("  " + "-".repeat(50));
        for (int i = 0; i < employees.size(); i++) {
            Employee employee = employees.get(i);
            System.out.printf("    %-4s %-25s %-20s%n",
                (i + 1) + ".",
                employee.getEmployeeName(),
                employee.getDepartmentName());
        }
        System.out.println("  " + (employees.size() + 1) + ".
Cancel");
        System.out.println("  " + "-".repeat(50));
        System.out.print("  Select an employee: ");

        String choice = scanner.nextLine().trim();
        try {
            int index = Integer.parseInt(choice) - 1;
            if (index == employees.size()) return;
            if (index < 0 || index >= employees.size()) {
                System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);
                router.pause();
                return;
            }
            showScheduleForm(employees.get(index));
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid selection." +
ANSI_RESET);
            router.pause();
        }
    }

    private void showScheduleForm(Employee employee) {
        boolean entering = true;
        while (entering) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + "  Scheduling: " +
employee.getEmployeeName() + ANSI_RESET);
            System.out.println("  " + "-".repeat(34));
            System.out.println("  Enter C at any prompt to cancel.");
        }
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
System.out.println("  " + "-".repeat(34));

String date = readLine("  Date (MM/dd/yyyy): ");
if (isCancel(date)) return;

String start = readLine("  Start Time (HH:mm): ");
if (isCancel(start)) return;

String end = readLine("  End Time (HH:mm): ");
if (isCancel(end)) return;

String result =
schedulingController.manageEmployeeSchedule(
    employee.getId(), date, start, end);

    if (result.contains("successfully")) {
        System.out.println(ANSI_GREEN + "  " + result +
ANSI_RESET);
    } else {
        System.out.println(ANSI_RED + "  " + result +
ANSI_RESET);
    }
    router.pause();

    if (result.contains("successfully")
        || result.equals("Employee does not exist.")
        || result.equals("Unable to save schedule because
of a file error.)) {
        entering = false;
    }
}

private void showViewSchedules() {
    router.clearScreen();
    router.printBanner();
    List<EmployeeSchedule> schedules;
    try {
        schedules = scheduleFileHandler.readAllSchedules();
    } catch (Exception e) {
        System.out.println(ANSI_RED + "  Could not load
schedules." + ANSI_RESET);
        router.pause();
        return;
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
        if (schedules.isEmpty()) {
            System.out.println(" No schedules found.");
            router.pause();
            return;
        }

        System.out.println(ANSI_BOLD + " Employee Schedules" +
ANSI_RESET);
        System.out.println(" " + "-".repeat(74));
        System.out.printf(" %-20s %-22s %-12s %-10s %-10s%n",
            "Employee", "Department", "Date", "Start", "End");
        System.out.println(" " + "-".repeat(74));
        for (EmployeeSchedule schedule : schedules) {
            System.out.printf(" %-20s %-22s %-12s %-10s %-10s%n",
                schedule.getEmployeeName(),
                schedule.getDepartmentName(),
                schedule.getScheduleDate(),
                to12Hour(schedule.getStartTime()),
                to12Hour(schedule.getEndTime()));
        }
        System.out.println(" " + "-".repeat(74));
        router.pause();
    }

    private String readLine(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    private boolean isCancel(String value) {
        return value.equalsIgnoreCase("C") ||
value.equalsIgnoreCase("CANCEL");
    }

    private String to12Hour(String time24) {
        String[] parts = time24.split(":");
        int hour = Integer.parseInt(parts[0]);
        int min = Integer.parseInt(parts[1]);
        String amPm = hour < 12 ? "AM" : "PM";
        if (hour == 0) hour = 12;
        else if (hour > 12) hour -= 12;
        return String.format("%d:%02d %s", hour, min, amPm);
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

Iteration 2 Implementation - ITER 1 Updated User Interface

```
package ui;

import models.Item;
import models.ItemConfirmation;

import java.util.List;
import java.util.Scanner;

public class ValidationUI {
    private final UserInterface router;
    private final Scanner scanner;
    private final ItemConfirmation itemConfirmation;

    private static final String ANSI_BOLD = "\u001B[1m";
    private static final String ANSI_RESET = "\u001B[0m";
    private static final String ANSI_RED = "\u001B[31m";
    private static final String ANSI_GREEN = "\u001B[32m";

    public ValidationUI(UserInterface router, Scanner scanner) {
        this.router = router;
        this.scanner = scanner;
        this.itemConfirmation = new ItemConfirmation();
    }

    public void showValidationMenu() {
        boolean running = true;
        while (running) {
            router.clearScreen();
            router.printBanner();
            System.out.println(ANSI_BOLD + " Item Validation" +
ANSI_RESET);
            System.out.println(" " + "-".repeat(34));
            System.out.println(" 1. Validate Item");
            System.out.println(" 2. Back");
            System.out.println(" " + "-".repeat(34));
            System.out.print(" Select an option: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1": showItemSelection(); break;
                case "2": running = false; break;
                default:
                    System.out.println(ANSI_RED + " Invalid option."
+ ANSI_RESET);

                    router.pause();
            }
        }
    }
}
```

Iteration 2 Implementation - ITER 1 Updated User Interface

```
    }
}

private void showItemSelection() {
    List<Item> items =
itemConfirmation.getDataProvider().getAllItems();
    List<String> ids =
itemConfirmation.getDataProvider().getAllIds();

    boolean running = true;
    while (running) {
        router.clearScreen();
        router.printBanner();
        System.out.println(ANSI_BOLD + "  Select Item to Validate"
+ ANSI_RESET);
        System.out.println("  " + "-".repeat(60));
        System.out.printf("  %-4s %-25s %-20s %-10s%n", "No.",
"Name", "Location", "Status");
        System.out.println("  " + "-".repeat(60));
        for (int i = 0; i < items.size(); i++) {
            Item item = items.get(i);
            System.out.printf("  %-4s %-25s %-20s %-10s%n",
                (i + 1) + ".",
                item.getName(),
                item.getLocation().name().replace("_", " "),
                item.getStatus());
        }
        System.out.println("  " + (items.size() + 1) + ". Back");
        System.out.println("  " + "-".repeat(60));
        System.out.print("  Select an item: ");

        String choice = scanner.nextLine().trim();
        int selection;
        try {
            selection = Integer.parseInt(choice);
        } catch (NumberFormatException e) {
            System.out.println(ANSI_RED + "  Invalid input." +
ANSI_RESET);
            router.pause();
            continue;
        }

        if (selection == items.size() + 1) {
            running = false;
        }
    }
}
```


Iteration 2 Implementation - ITER 1 Updated User Interface

```
        } else if (selection >= 1 && selection <= items.size()) {
            validateItem(ids.get(selection - 1));
        } else {
            System.out.println(ANSI_RED + "    Invalid selection." +
ANSI_RESET);
            router.pause();
        }
    }
}

private void validateItem(String itemId) {
    router.clearScreen();
    router.printBanner();
    boolean ok      = itemConfirmation.validateItem(itemId);
    String result = itemConfirmation.getGenMessage();
    Item item      = itemConfirmation.getItem();

    System.out.println(ANSI_BOLD + "    Validation Result" +
ANSI_RESET);
    System.out.println("    " + "-".repeat(40));
    if (ok) {
        System.out.println(ANSI_GREEN + "    " + result +
ANSI_RESET);
        if (item != null) System.out.println("    " +
item.getInfo());
    } else {
        System.out.println(ANSI_RED + "    " + result + ANSI_RESET);
    }
    System.out.println("    " + "-".repeat(40));
    router.pause();
}
}
```